SIMATS ENGINEERING

Department of Computer Science and Engineering CSA16 – Data Warehousing and Data Mining ASSIGNMENT 1 – CO1

Design, Build and Validate a Data Warehouse and OLAP Reporting Solution for a Retail Chain

CO1: Model the data for a data warehouse and apply the various operations that can be performed on the data sets.

Student Name	Register Number	Department / Section	Academic Year / Semester	Faculty Name	Nature of Assignment	Duration	Total Marks	Date of Submission	Date of Evaluation
G Prashanth	192311366	CSE	2026–2027	Dr.D. Beulah David	Individual	1 Week	50	06/11/2026	

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max	Marks Awarded	Remarks / Evidence Expected
1	Decision-Support Requirement	4		At least eight strategic/tactical business questions; measures and dimensions; FreshCart-specific OLTP vs DSS justification.
2	Warehouse Architecture	3		Labelled architecture showing sources, staging, ETL, warehouse, metadata, data marts, OLAP and reporting tools; justified architecture choice.
3	Star Schema Design	6		Declared grain; fact measures and additivity; dimensions and hierarchies; keys; conformed Returns fact; schema diagram and DDL.
4	Snowflake Schema Comparison	4		Product and Store snowflake designs; storage calculation; join counts for three queries; performance, ETL and usability comparison; justified choice.
5	ETL Process Design	5		Data-quality issues and transformations; surrogate-key lookup; unknown/late dimensions; SCD choice; load sequence; rejected rows/audit; before-and-after extract.
6	Data Cube and OLAP Operations	5		Cuboid calculations with/without hierarchies; base/apex cuboids; roll-up, drill-down, slice, dice and pivot with FreshCart questions and SQL; materialisation choice.
7	OLAP Application Development	5		Working interactive OLAP application; documented data generation and scale; all five OLAP operations; screenshots, source code and reconciliation.
8	Query Processing Performance	4		Three queries tested on OLTP, star schema and indexed/summary design; execution time, rows read, joins, query plans, storage/load cost and interpretation.
9	KNIME Reporting Workflow	3		Annotated KNIME workflow; node purposes and ETL/OLAP mapping; exported report; limitation at 1,000,000+ rows.
10	Final Design Recommendation	3		Evidence-based recommendation of architecture/schema; comparison with alternatives; trade-offs, load cost, limitations and scalability.
11A	Design Challenge – Impact Analysis	2		Identifies affected and unaffected schema, ETL, cube and architecture components; distinguishes structural, loading and performance impacts.
11B	Design Challenge – Evaluate Original Design	2		Tests original design against 20,000 daily online orders and hourly refresh; identifies adequate areas and limitations.
11C	Design Challenge – Adaptation or Redesign	2		Presents revised schema/ETL flow, product-code mapping, delivery-partner attribute, SCD treatment and incremental/micro-batch loading or aggregates.
11D	Design Challenge – Evidence-Based Defence	2		Supports decision with measured/calculated load time, query time and storage; identifies deployment monitoring metrics.
	TOTAL	**50**		**Marks to be awarded by the faculty.**

OVERALL REMARKS

Faculty feedback:

_____

_____

Student Signature	Faculty Signature	Program Director
		_

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = present P = partly present ✗ = missing. Mark the status after checking the student's submitted report and supporting evidence.

Task	Evidence Expected	Status	Note
1	Business questions, measures/dimensions and OLTP–DSS analysis	☐	At least eight questions; FreshCart-specific explanation.
2	Warehouse architecture diagram and justification	☐	All required layers, tools and architecture type shown.
3	Star schema, grain, measures, dimensions, keys and DDL	☐	Sales and Returns facts; conformed dimensions demonstrated.
4	Snowflake schema and quantitative comparison	☐	Storage calculations, join counts and design decision included.
5	ETL flow, cleaning rules, SCD decision and cleaned data	☐	All sample defects addressed; rejected rows and audit included.
6	Cube lattice calculations and five OLAP operations	☐	Business questions, cuboids and equivalent SQL included.
7	Working OLAP application and validation evidence	☐	Data sizes, user-selected operations, screenshots, code and reconciliation.
8	Query performance results and query plans	☐	Three queries across three designs; time, rows, joins and storage/load cost.
9	KNIME workflow and exported management report	☐	Annotated nodes and ETL/OLAP concept mapping; limitation discussed.
10	Final design recommendation	☐	Supported by the student's comparisons and measured results.
11A	Impact analysis of changed requirements	☐	Affected and unaffected components identified.
11B	Evaluation under new online-channel workload	☐	20,000 orders/day and hourly refresh considered/tested.
11C	Revised design or adaptation	☐	New attribute, code mapping, SCD and load strategy addressed.
11D	Evidence-based defence and monitoring plan	☐	Measurements/calculations and monitoring metrics included.

IMPLEMENTATION EVIDENCE CHAIN (Tasks 7–9)

Evidence Item	Status	Observation
Warehouse schema and DDL	☐	Star/snowflake structures, keys, constraints and declared grain are documented.
Data source and generation	☐	Real or synthetic data source and generation method documented for 1,000 to 1,000,000+ fact rows.
Application source code	☐	Readable OLAP application code, language, dependencies and execution instructions provided.
Interactive OLAP inputs	☐	User can select operation, dimension, hierarchy level and filter values.
Expected and actual results	☐	Outputs for roll-up, drill-down, slice, dice and pivot are recorded and interpreted.
Reconciliation evidence	☐	Independent totals agree with source data or differences are explained.
Performance test evidence	☐	Queries, data sizes, execution times, rows read, joins and query plans recorded.
Index / summary structure evidence	☐	Additional storage and load cost are recorded and trade-offs explained.
KNIME workflow evidence	☐	Workflow screenshot, node descriptions and exported report included.
Reproducibility and consistency	☐	Code, data, screenshots, tables and report values are consistent and reproducible.

SUBMISSION CHECKLIST (not separately marked)

Requirement	Status
All Tasks 1–10 and Day-4 Design Challenge 11A–11D answered	☐ Yes ☐ Partly ☐ No
Business questions, architecture and dimensional model are FreshCart-specific	☐ Yes ☐ Partly ☐ No
Fact-table grain, measures, dimensions and keys are clearly stated	☐ Yes ☐ Partly ☐ No
Star and snowflake designs are compared using calculations and examples	☐ Yes ☐ Partly ☐ No
ETL transformations, SCD handling and cleaned sample data are shown	☐ Yes ☐ Partly ☐ No
All five OLAP operations are demonstrated with relevant queries	☐ Yes ☐ Partly ☐ No
OLAP application, source code, screenshots and reconciliation are provided	☐ Yes ☐ Partly ☐ No
Query performance tests include plans, rows read and storage/load costs	☐ Yes ☐ Partly ☐ No
KNIME workflow and exported report are included	☐ Yes ☐ Partly ☐ No
Day-4 challenge addresses online orders, hourly refresh and changing customer details	☐ Yes ☐ Partly ☐ No
All diagrams, tables, calculations and references are numbered and readable	☐ Yes ☐ Partly ☐ No
No unfilled template placeholders remain in the student report	☐ Yes ☐ Partly ☐ No
Viva recommended where implementation, calculations or results need clarification	☐ Yes ☐ No

Note: This DWDM assignment evaluation template is based on Assignment 1 – CO1. Blank marks and evidence statuses are intentionally left for faculty evaluation.

Task 1 – Analyse the Decision-Support Requirement (4 Marks)

QUESTION

Examine why FreshCart's existing operational system is unsuitable for management analysis, and define what the warehouse must support. Discuss:

- At least eight business questions (including the five given) classified as strategic or tactical
- For each question, the measures and dimensions it requires
- How the operational system and a decision-support system differ in purpose, data currency, structure, query pattern, users and volume
- Which of the questions cannot be answered efficiently from the OLTP database, and why

Do not simply provide a table of textbook differences; relate each difference to FreshCart's situation.

ANSWER

1.1 Working assumptions about FreshCart's volume

The brief gives 120 supermarkets but no transaction volume. I assume about 3,000 bills per store per day and about 4 lines per bill, which gives roughly **1.4 million sales lines per day and about 525 million per year**. This is an assumption, and it is used only to scale the measurements in later tasks. The experiments in this report test 1,000 to 1,000,000 lines, which is therefore about one-fifth of one percent of a single year of real volume.

1.2 Ten business questions, classified, with measures and dimensions

Strategic questions support long-horizon decisions by senior management, such as range, store network and pricing. **Tactical** questions support weekly or daily decisions by category managers, store managers and marketing, such as replenishment, staffing and campaign tuning.

#	Business question	Type	Measures	Dimensions (level)
Q1	Which product categories show declining net sales, quarter over quarter, in each region?	Strategic	Net sales; QoQ % change (derived)	Date (quarter), Product (category), Store (region)
Q2	Which promotions increased the average basket value, and in which stores?	Tactical	Net sales, distinct order count, average basket value = net sales ÷ orders (non-additive)	Promotion, Store (store/state), Date (month)
Q3	How does weekend sales performance differ from weekday performance for each department?	Tactical	Net sales, quantity, net sales per day	Date (day type: weekday/weekend), Product (department)
Q4	Which customer segments return the most items, and of which products?	Tactical	Quantity returned, refund amount, return rate = returned ÷ sold (non-additive)	Customer (segment), Product (product/category), Date (month); uses Returns and Sales facts
Q5	What is the year-to-date sales of every store compared with the same period last year?	Strategic	Net sales YTD, net sales same-period-last-year, growth %	Store (store), Date (day → year, with a 'same day last year' attribute)
Q6	Which customer segments contribute most of the net-sales growth year on year?	Strategic	Net sales, growth %	Customer (segment), Date (year, quarter)
Q7	Which stores give away more than 15% of gross sales as discount this month?	Tactical	Discount amount, gross amount, discount % (non-additive)	Store (store), Promotion, Date (month)

#	Business question	Type	Measures	Dimensions (level)
Q8	Which are the top 20 products by quantity in each department over the last 7 days (for replenishment)?	Tactical	Quantity, net sales	Product (product, department), Date (day), Store (region)
Q9	What share of total net sales does each region contribute, year by year?	Strategic	Net sales, share of total (derived)	Store (region), Date (year)
Q10	Which stores have the highest return rate this month?	Tactical	Quantity returned, quantity sold, return rate	Store (store), Date (month); Returns and Sales facts

Together, the ten questions use five dimensions (Date, Product, Store, Customer, Promotion) and two business processes (Sales and Returns). This is why the warehouse is designed around **two fact tables that share conformed dimensions** (Task 3). Only the measures net sales, gross sales, discount and quantity need to be stored. Ratios (basket value, return rate, discount %, growth %) must always be recomputed from additive components, because averaging ratios gives wrong totals when the query is rolled up.

1.3 Why FreshCart's OLTP system is unsuitable, difference by difference

Aspect	FreshCart today (OLTP)	FreshCart needs (warehouse)	Consequence at FreshCart
Purpose	Ring up a sale in seconds, record a return, swipe a loyalty card, apply a promotion. The design goal is a fast, correct single-transaction write.	Explain performance: trends, comparisons, promotion effectiveness.	The tables Orders, OrderLines, Products, Stores and others are split to avoid update anomalies. That is exactly what makes analytic questions need many joins.
Data currency	Current state only. A customer's address and loyalty tier are overwritten when they change. Prices and promotions are the ones valid now.	Time-variant history: the address and tier that applied at the time of each order, and promotions as they ran.	Q4 and Q6 (segment-based) cannot be answered correctly, because a customer who moved from Gold to Platinum appears as Platinum for all past sales. Section 11 quantifies this: **9.98% of historic net sales are misattributed** when 10% of customers change tier.
Structure	Normalised (3NF), about 9 core tables. Product → brand → category → department and store → city → state → region are separate tables.	Subject-oriented star schema with denormalised dimensions and one agreed definition of each measure.	Q1 needs **8 joins** on OLTP and 3 on the star. Departments also compute 'net sales' with different rules. The warehouse stores one governed definition: net sales = qty × unit price − line discount; refunds are held separately in the Returns fact.
Query pattern	Thousands of short indexed lookups and inserts per minute (WHERE order_id = ?). Each touches a few rows.	Few, long queries that scan and aggregate millions of rows and group by several dimension levels.	An analyst's Q1 reads every sales line for two years. On the 1M-row test it takes 1.98 s on a single idle machine. Run beside live billing at 525M rows, it would hold locks and buffer cache for many minutes, which is **why billing counters slow down at peak hours**.
Users	Thousands of cashiers and store staff, with predictable screens.	Tens of analysts, category managers and executives, with unpredictable ad-hoc questions.	Many 'analyst' queries come from a handful of people but each is far heavier than a billing transaction, so a few users can degrade service for thousands.
Volume	Recent data (months) is kept online; old data is archived to keep the OLTP small and fast.	Years of history (at least 24 months for year-on-year Q5). It grows to 10^8–10^9 rows.	Q5 compares this year to last year, which the OLTP system has often archived. Keeping the history in OLTP would slow billing further.

1.4 Which questions cannot be answered efficiently from OLTP, and why

Question	Why OLTP fails	Evidence from this project
Q1 (declining categories QoQ by region)	Needs a join chain line → product → brand → category and line → order → store → city → state → region (8 joins), a full scan of all lines and a derived quarter from a date string.	1M lines: 1.98 s. Scaled linearly to 525M lines this is about 17 minutes, while competing with billing.
Q2 (average basket by promotion and store)	Needs two-level aggregation (sum per order, then average per promotion and store). Promotion rules are held in a separate table and a basket is only defined after summing lines.	5 joins and a sub-aggregation; 0.72 s at 1M lines.
Q4 / Q6 (segments)	Segment and tier are overwritten, so history is lost. Returns must be linked back to the original sale line to reach the product and customer.	Section 11 test: 9.98% of net sales get the wrong tier.
Q5 (YTD vs same period last year)	No calendar logic in OLTP (fiscal year, 'same day last year'). Last year's rows may be archived. A union of two date ranges over every store is required.	Needs a Date dimension and at least 24 months of history.
Q9 (regional share)	Needs a grand total as the denominator and a region total as the numerator over the same scan, which is a window or two-pass calculation over all lines.	Can be read from the quarter × region × category summary table in about 1 ms (Task 8).
Q3, Q7, Q8, Q10	Answerable on OLTP for small volumes, but each is a scan-and-aggregate on the live billing tables.	Q3: 1.61 s at 1M lines on OLTP versus 0.97 s on the star and under 1 ms on a summary table.

1.5 What the warehouse must support

- **One version of the truth:** a single, governed definition of net sales, gross sales, discount and returns, held in metadata.
- **Time-variant, non-volatile history** of at least two years, with changes to customer address and tier preserved (slowly changing dimensions).
- **Conformed dimensions** (Date, Product, Store, Customer, Promotion) so that Sales and Returns can be compared (drill-across).
- **Multi-level aggregation:** roll-up, drill-down, slice, dice and pivot along the hierarchies day → month → quarter → year, product → category → department, store → city → state → region.
- **Isolation from billing:** analytic queries must run on a separate server and never touch the OLTP tables at peak hours.
- **Scalability:** practical from 1,000 to 1,000,000,000 lines, which requires pre-aggregation whose storage and refresh cost is planned (Tasks 6 and 8).
- **Data quality and audit:** every loaded or rejected row is accounted for (Task 5).

Task 2 – Propose the Warehouse Architecture (3 Marks)

QUESTION

Propose an architecture for FreshCart and represent it using a labelled diagram. It must show:

- Data sources, staging area, ETL process, warehouse storage and metadata repository
- Data marts, if any, and the OLAP server
- Front-end tools for reporting, query and analysis
- Your choice among an enterprise warehouse, independent data marts, dependent data marts or a virtual warehouse, and why

ANSWER

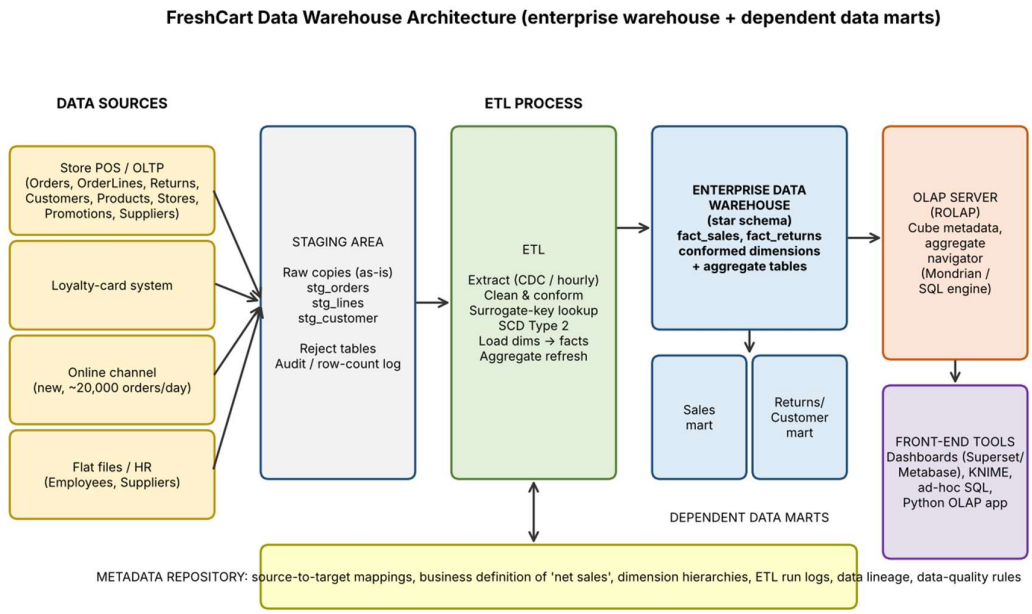


Figure 2.1 – Proposed FreshCart architecture (enterprise warehouse with dependent data marts and a ROLAP server)

2.1 Components and how each is shaped by FreshCart's needs

Component	What it contains / does at FreshCart	Why it is needed here
Data sources	Store POS/OLTP (Orders, OrderLines, Customers, Products, Stores, Promotions, Returns), the loyalty-card system, HR/supplier flat files, and later the online channel feed (Task 11).	Each source uses different codes and formats (e.g. Chennai-04 vs chennai-04); they must be integrated.
Staging area	Raw, unmodified copies of each extract plus a batch_id, reject tables and the audit log. It is not queryable by users.	Isolates the OLTP from heavy cleaning work. Extraction is quick and cleansing is slow, so the two are decoupled.
ETL process	Extract (full first, then by watermark or CDC), clean, conform, surrogate-key lookup, SCD Type 2, load dimensions then facts, refresh aggregates.	Implements the single definition of net sales and fixes defects such as mixed date formats and currencies (Task 5).
Warehouse storage	Star schema: fact_sales, fact_returns, conformed dimensions, plus aggregate (summary) tables.	Gives fast scan-and-aggregate queries (Task 8).
Dependent data marts	Sales mart (fact_sales with Date, Product, Store, Promotion) and a Returns/Customer mart (fact_returns with Customer, Product, Store). Both are logical subsets or views over the enterprise warehouse.	Departments get a simple, focused view without owning separate copies of the data.

Component	What it contains / does at FreshCart	Why it is needed here
OLAP server (ROLAP)	Translates cube operations into SQL on the star schema and aggregate tables, and keeps the cube metadata (dimensions, hierarchies, measures).	ROLAP is chosen over MOLAP because Product (50,000+ members) and Customer are large and sparse. A MOLAP cube would also have to be rebuilt on each hourly refresh (Task 11).
Metadata repository	Business definitions (net sales), hierarchy definitions, source-to-target mapping, data-quality rules, ETL run history and lineage.	Ends the 'different departments, different net sales' problem and lets auditors trace any figure to its source.
Front-end tools	Dashboards (Apache Superset or Metabase), KNIME workflows (Task 9), ad-hoc SQL, and the Python OLAP application (Task 7).	Different users need different tools: executives want dashboards, analysts want ad-hoc queries.

2.2 Choice of architecture type

Option	How it would work at FreshCart	Verdict
Independent data marts	Each department (merchandising, marketing, finance) builds its own mart from the sources.	**Rejected.** This reproduces today's problem. Each mart would apply its own net-sales rule and dimensions would not agree, so figures would not reconcile.
Virtual warehouse	Views over the live OLTP database.	**Rejected.** There is no isolation, so billing still slows down, no history is kept, and Q1 would take about 17 minutes.
Dependent data marts only	Marts fed from a central warehouse.	**Part of the answer.** The marts are the departmental windows onto the central store.
Enterprise data warehouse	One integrated, conformed, historical warehouse that is the single source of truth.	**Chosen** as the core, with dependent marts on top.

Decision: an enterprise data warehouse with dependent data marts and a ROLAP server. Three FreshCart facts justify it. (1) Departments currently disagree about net sales, so the first requirement is a *single conformed* definition, which only a central store with conformed dimensions provides. (2) Billing must be protected, so the warehouse runs on a separate server fed only by the staging area. (3) Sales and Returns must be analysed together (Q4, Q10), and that is possible only if both facts share the same conformed dimensions. The marts add departmental simplicity without extra copies of truth. Because it is built on conformed dimensions, a new source (the online channel) later plugs in with only additive changes (Task 11).

Task 3 – Design the Star Schema (6 Marks)

QUESTION

Design a dimensional model for FreshCart's sales analysis using a star schema. You must:

- State the business process and declare the grain before listing measures
- Identify the fact table and measures, classified as additive, semi-additive or non-additive
- Identify the dimension tables (at minimum Date, Product, Store, Customer, Promotion) with attributes and concept hierarchies
- Define primary, surrogate and foreign keys
- Show how a second fact table (e.g. Returns) shares conformed dimensions with Sales
- Provide the schema diagram and the DDL statements; show every measure and dimension is consistent with the grain.

ANSWER

3.1 Business process and declared grain

Business process: retail sales at the point of sale (billing), later extended with online orders.

Declared grain of fact_sales: one row per product (SKU) on one order line of one order, at one store, on one date. Exactly one row exists for every row of the OLTP OrderLines table.

3.2 Star schema diagram

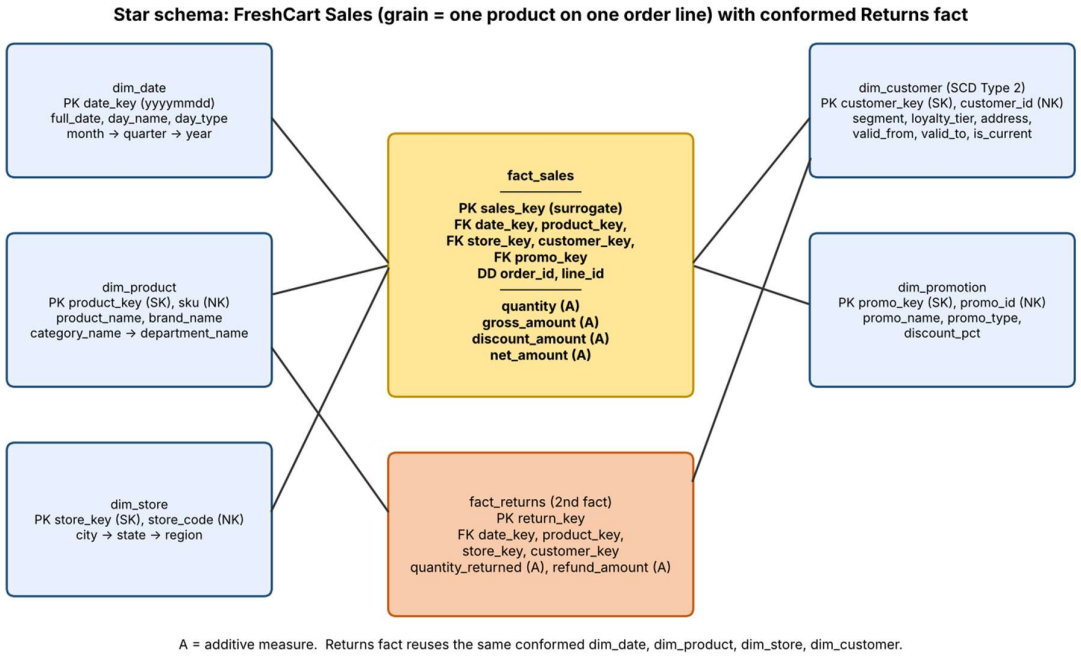


Figure 3.1 – Star schema for FreshCart sales, with the conformed Returns fact. (A) = additive.

3.3 Measures and their additivity

Measure	Definition	Additivity	Consistent with grain?
quantity	Units sold on the line	Additive across all dimensions	Yes, one quantity per line.
gross_amount	quantity × unit price	Additive	Yes, a line-level amount.

Measure	Definition	Additivity	Consistent with grain?
discount_amount	Promotion discount applied to the line	Additive	Yes. The order-level promotion is allocated to lines.
net_amount	gross_amount – discount_amount (**the governed 'net sales'**)	Additive	Yes.
unit price, discount %	Derived ratios (gross ÷ qty; discount ÷ gross)	**Non-additive.** Never summed; recomputed from additive parts.	Not stored; derived at query time.
average basket value	Net sales ÷ distinct orders	**Non-additive** (average of a distinct count)	Not stored. Order count uses the degenerate dimension order_id.
return rate	Quantity returned ÷ quantity sold	**Non-additive** ratio	Computed across two facts by drill-across.
Stock on hand / loyalty-point balance	A balance at a point in time	**Semi-additive** (sum across stores, but not across time; use average or last value over time)	Not in fact_sales, because it is not a line-level event. It would live in a periodic-snapshot fact (e.g. fact_inventory_daily) that reuses Date, Product and Store.

3.4 Dimensions, attributes and hierarchies

Dimension	Key (surrogate / natural)	Key attributes	Concept hierarchy
dim_date	date_key = yyyymmdd (smart integer) / full_date	day_name, is_weekend, day_type, month_no, month_name, year_month, quarter_no, year_quarter, year_no	day → month → quarter → year (and day → week; fiscal year if required)
dim_product	product_key (SK) / sku, product_id (NK)	product_name, brand_name, category_name, department_name, unit_list_price	product → brand → category → department
dim_store	store_key (SK) / store_code (NK)	store_name, city_name, state_name, region_name	store → city → state → region
dim_customer (SCD Type 2)	customer_key (SK) / customer_id (NK)	customer_name, segment, loyalty_tier, address (city), valid_from, valid_to, is_current	segment (no hierarchy); city → state for addresses
dim_promotion	promo_key (SK) / promo_id (NK)	promo_name, promo_type, discount_pct	promotion → promo_type

3.5 Keys

- **Surrogate keys (SK)** such as product_key are meaningless integers generated by the warehouse. They insulate the warehouse from source-system code changes (the online channel will use a different product code, see Task 11), they allow several versions of the same customer (SCD Type 2), and they allow 'Unknown' members (key −1).
- **Natural/business keys (NK)** such as sku or customer_id are kept in the dimension for lookup during ETL.
- **Primary keys:** each dimension's SK; fact_sales.sales_key (SK); fact_returns.return_key.
- **Foreign keys:** fact_sales.date_key, product_key, store_key, customer_key, promo_key point to the dimensions. The full set of foreign keys together uniquely identifies the grain only with the degenerate keys, so (order_id, line_id) carries a UNIQUE constraint.

- **Degenerate dimensions:** order_id and line_id are stored in the fact with no dimension table. They allow order counts and drill-through to the source.

3.6 Second fact table sharing conformed dimensions

fact_returns has its own grain: *one row per returned line item*. It reuses the **same** dim_date, dim_product, dim_store and dim_customer tables (same keys, same attributes, same hierarchies). It does not use dim_promotion, because the promotion belongs to the original sale, and it carries line_id so that a return can be traced back to the original sale. Because the dimensions are conformed, a **drill-across** query can compare Sales and Returns by category and quarter: aggregate each fact separately to the same grain, then join the two result sets on the shared attributes. This avoids the fan-out error that would result from joining the two fact tables directly.

```
-- Drill-across: return rate by category and quarter (Sales and Returns share conformed dimensions)
WITH s AS (SELECT p.category_name, d.year_quarter, SUM(f.quantity) AS qty_sold
           FROM fact_sales f JOIN dim_product p USING(product_key) JOIN dim_date d USING(date_key)
           GROUP BY 1,2),
     r AS (SELECT p.category_name, d.year_quarter, SUM(f.quantity_returned) AS qty_returned
           FROM fact_returns f JOIN dim_product p USING(product_key) JOIN dim_date d
USING(date_key)
           GROUP BY 1,2)
SELECT s.category_name, s.year_quarter, s.qty_sold, COALESCE(r.qty_returned,0) AS qty_returned,
       ROUND(100.0*COALESCE(r.qty_returned,0)/s.qty_sold,2) AS return_rate_pct
FROM s LEFT JOIN r USING(category_name, year_quarter);
```

3.7 DDL statements

```
CREATE TABLE dim_date (
  date_key      INTEGER      PRIMARY KEY,          -- yyyymmdd
  full_date     DATE         NOT NULL UNIQUE,
  day_name      VARCHAR(9)   NOT NULL,
  is_weekend    SMALLINT     NOT NULL CHECK (is_weekend IN (0,1)),
  day_type      VARCHAR(7)   NOT NULL,             -- 'Weekday' / 'Weekend'
  month_no      SMALLINT     NOT NULL,
  month_name    VARCHAR(9)   NOT NULL,
  year_month    CHAR(7)      NOT NULL,             -- '2026-11'
  quarter_no    SMALLINT     NOT NULL,
  year_quarter  CHAR(7)      NOT NULL,             -- '2026-Q4'
  year_no       SMALLINT     NOT NULL
);
CREATE TABLE dim_product (
  product_key     INTEGER      PRIMARY KEY,        -- surrogate
  sku             VARCHAR(12)  NOT NULL,           -- natural key
  product_name    VARCHAR(40)  NOT NULL,
  brand_name      VARCHAR(30)  NOT NULL,
  category_name   VARCHAR(30)  NOT NULL,
  department_name VARCHAR(25)  NOT NULL,
  unit_list_price NUMERIC(10,2)
);
CREATE TABLE dim_store (
  store_key   INTEGER     PRIMARY KEY,
  store_code  VARCHAR(10) NOT NULL UNIQUE,         -- natural key, e.g. 'Chennai-04'
  store_name  VARCHAR(30),
  city_name   VARCHAR(30) NOT NULL,
  state_name  VARCHAR(25) NOT NULL,
  region_name VARCHAR(15) NOT NULL
);
CREATE TABLE dim_customer (                         -- SCD Type 2
  customer_key  INTEGER     PRIMARY KEY,           -- surrogate, one per VERSION
  customer_id   VARCHAR(12) NOT NULL,              -- natural key (loyalty id)
  customer_name VARCHAR(40) NOT NULL,
  segment       VARCHAR(10) NOT NULL,
  loyalty_tier  VARCHAR(10),
  city_name     VARCHAR(30),
  valid_from    DATE        NOT NULL,
  valid_to      DATE        NOT NULL DEFAULT '9999-12-31',
  is_current    SMALLINT    NOT NULL DEFAULT 1
);
CREATE INDEX ix_cust_nk ON dim_customer(customer_id, valid_from);
```

```
CREATE TABLE dim_promotion (
  promo_key    INTEGER      PRIMARY KEY,
  promo_id     VARCHAR(10)  NOT NULL,
  promo_name   VARCHAR(40)  NOT NULL,
  promo_type   VARCHAR(15),
  discount_pct NUMERIC(5,2)
);
CREATE TABLE fact_sales (                          -- grain: one row per order line
  sales_key       BIGINT   PRIMARY KEY,
  order_id        INTEGER  NOT NULL,               -- degenerate dimension
  line_id         INTEGER  NOT NULL,               -- degenerate dimension
  date_key        INTEGER  NOT NULL REFERENCES dim_date(date_key),
  product_key     INTEGER  NOT NULL REFERENCES dim_product(product_key),
  store_key       INTEGER  NOT NULL REFERENCES dim_store(store_key),
  customer_key    INTEGER  NOT NULL REFERENCES dim_customer(customer_key),
  promo_key       INTEGER  NOT NULL REFERENCES dim_promotion(promo_key),
  quantity        INTEGER       NOT NULL,          -- additive
  gross_amount    NUMERIC(12,2) NOT NULL,          -- additive
  discount_amount NUMERIC(12,2) NOT NULL,          -- additive
  net_amount      NUMERIC(12,2) NOT NULL,          -- additive
  UNIQUE (order_id, line_id)                       -- enforces the declared grain
);
CREATE TABLE fact_returns (                         -- grain: one row per returned line
  return_key        BIGINT  PRIMARY KEY,
  line_id           INTEGER NOT NULL,              -- links back to the sale
  date_key          INTEGER NOT NULL REFERENCES dim_date(date_key),      -- conformed
  product_key       INTEGER NOT NULL REFERENCES dim_product(product_key),-- conformed
  store_key         INTEGER NOT NULL REFERENCES dim_store(store_key),    -- conformed
  customer_key      INTEGER NOT NULL REFERENCES dim_customer(customer_key),-- conformed
  quantity_returned INTEGER       NOT NULL,
  refund_amount     NUMERIC(12,2) NOT NULL
);
-- 'Unknown' members for late or missing dimension values
INSERT INTO dim_product(product_key,sku,product_name,brand_name,category_name,department_name)
       VALUES (-1,'UNKNOWN','Unknown product','Unknown','Unknown','Unknown');
INSERT INTO dim_store(store_key,store_code,city_name,state_name,region_name)
       VALUES (-1,'UNKNOWN','Unknown','Unknown','Unknown');
INSERT INTO dim_customer(customer_key,customer_id,customer_name,segment,valid_from)
       VALUES (-1,'UNKNOWN','Unknown customer','Unknown','1900-01-01');
INSERT INTO dim_promotion(promo_key,promo_id,promo_name) VALUES (0,'NONE','No promotion');
```

The DDL above, with SQLite types, was executed unchanged in structure for all experiments in this report (see Appendix B for the build script).

Task 4 – Compare with a Snowflake Schema (4 Marks)

QUESTION

Create a snowflaked version of Product (product → brand → category → department) and Store (store → city → state → region). Assume 50,000 products, 800 brands, 60 categories, 12 departments, 120 stores, 70 cities, 6 states and 3 regions. Discuss:

- The additional tables, keys and joins introduced
- The estimated change in dimension storage (show the calculation)
- The number of joins for three representative queries in the star and snowflake versions
- Effect on query performance, ETL maintenance and ease of understanding
- Which design you would choose and under what condition you would change the decision

ANSWER

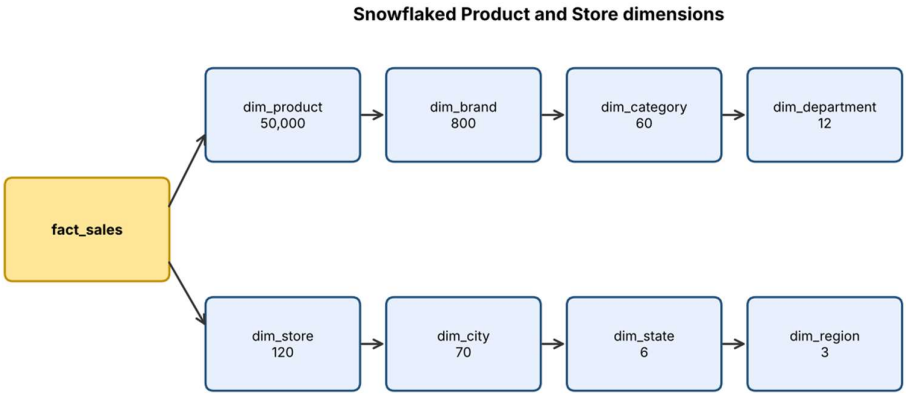


Figure 4.1 – Snowflaked Product and Store dimensions (row counts from the question)

4.1 Additional tables, keys and joins

New table	Primary key	Foreign key it holds	Rows
dim_brand	brand_key	category_key → dim_category	800
dim_category	category_key	department_key → dim_department	60
dim_department	department_key	—	12
dim_city	city_key	state_key → dim_state	70
dim_state	state_key	region_key → dim_region	6
dim_region	region_key	—	3

Six new tables are added. dim_product loses its three text columns and gains brand_key; dim_store loses city, state and region text and gains city_key. The number of dimension tables in the model rises from 5 to 11, and a path to the department needs 3 joins from the Product dimension (product → brand → category → department), while the star needed 0.

4.2 Storage estimate (calculation)

Assumed byte widths: integer key 4 B, SKU 12 B, product name 40 B, brand and category names 30 B each, department name 25 B, price 8 B, store code 10 B, store name 30 B, city name 30 B, state name 25 B, region name 15 B. (Row-overhead is ignored in both designs, so the comparison is fair.)

Table	Row width (bytes)	Rows	Total bytes
Star dim_product: 4+12+40+30+30+25+8	149	50,000	7,450,000
Star dim_store: 4+10+30+30+25+15	114	120	13,680
Star total			**7,463,680 (≈ 7.46 MB)**
Snowflake dim_product: 4+12+40+4 (brand_key)+8	68	50,000	3,400,000
dim_brand: 4+30+4	38	800	30,400
dim_category: 4+30+4	38	60	2,280
dim_department: 4+25	29	12	348
Snowflake dim_store: 4+10+30+4 (city_key)	48	120	5,760
dim_city: 4+30+4	38	70	2,660
dim_state: 4+25+4	33	6	198
dim_region: 4+15	19	3	57
Snowflake total			**3,441,703 (≈ 3.44 MB)**
Saving			**4,021,977 B ≈ 4.02 MB (53.9% of dimension storage)**

Put in context: the fact table is the real storage cost. Using 64 bytes per fact row (8 + 4 + 4 + five 4-byte keys + 4 + three 8-byte amounts), a 1,000,000-row fact table is 64 MB, so the 4.02 MB saving is only about **6.3% of the 1M-row fact table**, and about **0.06% at 100 million rows** (6.4 GB). The 'snowflake saves space' argument is therefore weak for FreshCart. (As a check, the measured SQLite star database at 1M rows is 57 MB.)

4.3 Join counts for three representative queries

Query	Star joins	Snowflake joins	Joins added
QA: net sales by region, category and quarter (Q1)	3 (Date, Product, Store)	8 = Date 1 + Product→Brand→Category 3 + Store→City→State→Region 4	+5
QB: average basket by promotion and state (Q2)	2 (Promotion, Store)	4 = Promotion 1 + Store→City→State 3	+2
QC: weekend vs weekday by department (Q3)	2 (Date, Product)	5 = Date 1 + Product→Brand→Category→Department 4	+3

Measured proxy. A fully normalised OLTP schema is effectively a fully snowflaked design (it needs 8, 5 and 5 joins for the same three queries). At 1,000,000 lines it took 1.98 s, 0.72 s and 1.61 s, against 1.37 s, 0.59 s and 0.96 s on the star. That is 1.45×, 1.23× and 1.67× slower. The penalty follows the number of extra joins, since each extra join adds one lookup per fact row. It is only a proxy, because the OLTP schema also has the extra Orders table.

4.4 Effects on performance, ETL and usability

Aspect	Star	Snowflake
Query performance	Fewer joins (3/2/2). Each dimension lookup is one probe per fact row.	More joins (8/4/5). Small lookup tables (3–800 rows) are cache-resident, so the cost is modest per join. The slowdown still grows with the fact table because the extra probes are paid per fact row, which matches the 1.2–1.7× measured proxy.
ETL maintenance	One load job per dimension. **A brand moving category must update about 62 product rows** (50,000 ÷ 800), and the repeated text risks inconsistency ('Beverages' vs 'beverages').	Six more load jobs and a strict load order (top levels first). **A brand moving category changes exactly 1 row.** Key lookups are needed at every level during ETL.
Ease of understanding	Business users see Product with brand, category and department in one table, and can drag them straight into a report.	Users must understand 11 tables and the join paths. BI tools need pre-built views to hide the complexity.
Hierarchy changes (SCD)	A reclassification touches many rows, and Type 2 history multiplies row versions.	A reclassification is a one-row change, and history is kept at the right level.

4.5 Choice for FreshCart, and the condition for changing it

Choice: star schema. The evidence is (a) a saving of only 4 MB, or about 6% of a 1M-row fact table and negligible beyond; (b) 3 joins instead of 8 for the flagship query; (c) a business-user audience that wants to drag-and-drop; and (d) product and store hierarchies that change rarely (a brand is reclassified occasionally, not daily).

I would change to a snowflake (or an outrigger for just the brand level) if: (1) the Product dimension grew to many millions of rows *and* hierarchy attributes were reclassified often, so that updating 62 rows per brand change (or mass SCD Type 2 versions) became an ETL bottleneck; (2) the same sub-dimension (e.g. Supplier or Brand) had to be shared by several fact tables and kept strictly consistent; or (3) the BI tool or governance rules required normalised reference data. For FreshCart none of these holds today.

Task 5 – Design the ETL Process (5 Marks)

QUESTION

The following extract from the source systems is to be loaded into the warehouse (Order ID, Customer, Store, Product, Qty, Unit Price, Order Date):

1001 | Rajesh Kumar | Chennai-04 | Basmati Rice 5kg | 2 | 620.00 | 03/11/2026

1002 | R. Kumar | chennai-04 | Basmati rice 5 kg | 1 | 620.00 | 2026-11-03

1003 | Anita Joseph | Pune-11 | Olive Oil 1L | -3 | 780.00 | 04-Nov-26

1004 | Meena S | (null) | Tea 250g | 4 | 95.50 | 04/11/2026

1005 | Anita Joseph | Pune-11 | Olive Oil 1L | 3 | 9.40 (USD) | 04/11/2026

1006 | Vikram Rao | Hyd-02 | (null) | 1 | 1,250 | 05/11/2026

Design the ETL process. Discuss every data-quality problem and its transformation rule; surrogate-key lookup and unknown / late-arriving members; the SCD technique for a change of customer address and why; initial vs incremental load and load order; rejected records and row-count audit. Show the before and after version of the extract.

ANSWER

5.1 ETL flow

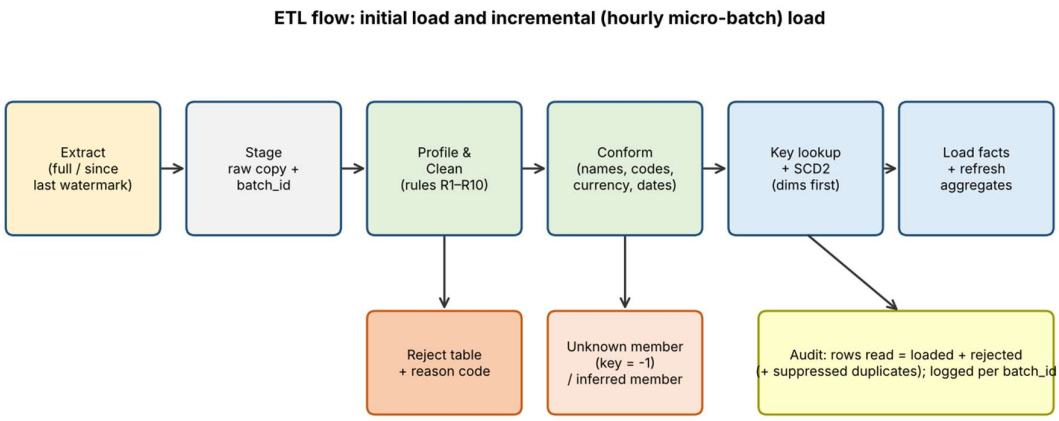


Figure 5.1 – ETL flow with reject, unknown-member and audit paths

5.2 Before: the raw extract (as received)

Order ID	Customer	Store	Product	Qty	Unit price	Order date
1001	Rajesh Kumar	Chennai-04	Basmati Rice 5kg	2	620.00	03/11/2026
1002	R. Kumar	chennai-04	Basmati rice 5 kg	1	620.00	2026-11-03
1003	Anita Joseph	Pune-11	Olive Oil 1L	-3	780.00	04-Nov-26
1004	Meena S	(null)	Tea 250g	4	95.50	04/11/2026
1005	Anita Joseph	Pune-11	Olive Oil 1L	3	9.40 (USD)	04/11/2026
1006	Vikram Rao	Hyd-02	(null)	1	1,250	05/11/2026

5.3 Every data-quality problem and the rule that resolves it

ID	Rows	Problem	Transformation rule
R1	1001–1006	**Three date formats** (dd/mm/yyyy, yyyy-mm-dd, dd-Mon-yy) and a two-digit year.	Parse by pattern into ISO yyyy-mm-dd, then to date_key (yyyymmdd). **dd/mm vs mm/dd ambiguity is settled by evidence:** 1002 says 2026-11-03 for the same customer, store and product as 1001 (03/11/2026), so the day comes first. Two-digit years map to 20yy. A date that fails to parse, or lies outside the dim_date range, is rejected.
R2	1002	**Customer name variant** 'R. Kumar' vs 'Rajesh Kumar'. The extract has no loyalty-card ID.	Standardise (trim, case, remove punctuation) and match on loyalty ID when present. With no ID, use a fuzzy rule (same surname, matching initial, same store) with a confidence flag. Here 'R. Kumar' is matched to Rajesh Kumar with flag MATCH_FUZZY and goes on a steward report. Recommendation to the business: capture the loyalty ID on every bill.
R3	1002	**Store code case** (chennai-04 vs Chennai-04).	Trim and standardise case, then look up in the store master by natural key store_code.
R4	1002	**Product name variants** ('Basmati rice 5 kg' vs 'Basmati Rice 5kg').	Normalise case and unit spacing ('5 kg' → '5kg'), then match to the product master and take the SKU. Names that do not match are sent for key lookup as unknown (see R6).
R5	1003	**Negative quantity** (−3) in a sales feed.	A sale must have qty > 0. If the source carries a return flag, the row is routed to fact_returns. Here there is no flag, and order 1005 (same customer, product and day) looks like a re-keyed version of the same sale, so 1003 is **rejected** with reason NEG_QTY and sent to the data steward. It is never silently reversed or 'fixed' by dropping the sign.
R6	1006	**Missing product** (null). A sale line must be for a product, otherwise the declared grain ('one product on one line') is broken.	**Reject** with reason MISSING_PRODUCT. The amount (1,250) is kept in the reject table so that it can be reprocessed when the source is corrected, and it appears in the reconciliation. A fallback 'Unknown product' member (−1) is used only when the product code is present but not yet in the master (late-arriving member).
R7	1004, 1006	**Missing store** (null) in 1004, and the literal text '(null)' rather than a real NULL.	Convert the text marker to NULL. For 1004, store is a descriptive dimension, not part of the grain, and the sale is valid, so **load with store_key = −1 ('Unknown store')**. Raise a DQ warning so the row can be corrected later, and net sales totals still reconcile.
R8	1005	**Wrong currency:** '9.40 (USD)' in an INR feed.	Split the amount and currency code, then convert with the exchange rate for the order date from a rate table: 9.40 × 83.00 = **780.20 INR**. Cross-check against the list price of Olive Oil 1L (780.00): a difference of 0.03% is within the 1% tolerance, so it passes. Store the original currency and rate for audit.
R9	1006	**Thousands separator in a number** ('1,250' held as text).	Remove the comma and cast to DECIMAL(12,2): 1250.00.
R10	1001/1002	**Possible duplicate sale:** same customer, store, product and day, with different order IDs.	Order IDs differ, so both are valid distinct orders and are loaded. A rule flags them as POSSIBLE_DUP for review (it never auto-deletes). Duplicate *order_id*s would be rejected by the UNIQUE(order_id, line_id) constraint.
R11	1003/1005	**Related anomalies** across rows: a negative-quantity row and a USD row for the same item, customer and day.	A cross-record check links them, so the steward report presents them together as 'probable same transaction, entered twice with different errors'. The warehouse takes 1005 and rejects 1003.

5.4 After: the cleaned extract and load outcome

Order ID	Customer (std)	Store (std)	Product (std)	Qty	Unit price (INR)	Order date	Outcome
1001	Rajesh Kumar	Chennai-04	Basmati Rice 5kg	2	620.00	2026-11-03	Loaded
1002	Rajesh Kumar (fuzzy match)	Chennai-04	Basmati Rice 5kg	1	620.00	2026-11-03	Loaded; warnings: MATCH_FUZZY, POSSIBLE_DUP
1003	Anita Joseph	Pune-11	Olive Oil 1L	−3	780.00	2026-11-04	**Rejected** (NEG_QTY)
1004	Meena S	**Unknown (−1)**	Tea 250g	4	95.50	2026-11-04	Loaded; warning: UNKNOWN_STORE

Order ID	Customer (std)	Store (std)	Product (std)	Qty	Unit price (INR)	Order date	Outcome
1005	Anita Joseph	Pune-11	Olive Oil 1L	3	780.20 (9.40 USD × 83.00)	2026-11-04	Loaded; warning: CURRENCY_CONVERTED
1006	Vikram Rao	Hyderabad-02	(missing)	1	1250.00	2026-11-05	**Rejected** (MISSING_PRODUCT)

Store 'Hyd-02' is expanded to the master code Hyderabad-02 through a code-mapping table. Without this, its lookup would also fail.

5.5 Surrogate-key lookup, unknown and late-arriving members

- **Lookup:** for every fact row, each business key is looked up in its dimension (date → date_key, SKU → product_key, store_code → store_key, customer → the customer **version valid on the order date** using valid_from ≤ order_date ≤ valid_to, promotion → promo_key). The lookup tables are cached in memory because the dimensions are small (Product 50,000 rows, Store 120).
- **Missing business key (NULL):** the fact is loaded with the **Unknown member** (key −1) only for descriptive dimensions that are not part of the grain (Store, Customer, Promotion → 0 'No promotion'). A missing Product or Date key is a grain violation and the row is rejected.
- **Late-arriving dimension member** (key present, but the master record has not arrived yet): create an **inferred member** with the natural key, placeholder attributes and inferred_flag = 1, and use its new surrogate key. When the real record arrives, overwrite the placeholder attributes in place (Type 1 on an inferred member). The fact rows never need re-keying.
- **Late-arriving fact** (an order arriving after the customer has changed address): the date-range lookup picks the version that was valid on the order date, not the current one.

5.6 Slowly changing dimension for a change of customer address

Decision: SCD Type 2 for address. A new row is inserted for the customer with a new surrogate key, the old row is closed (valid_to set, is_current = 0), and subsequent facts use the new key, while earlier facts keep the old key.

customer_key	customer_id	name	city	valid_from	valid_to	is_current
501	C1001	Rajesh Kumar	Chennai	2025-01-01	2026-08-14	0
9021	C1001	Rajesh Kumar	Pune	2026-08-15	9999-12-31	1

Technique	What it does	Verdict for address
Type 1 (overwrite)	Replaces the old address.	**Rejected.** History is rewritten, so past sales appear to come from the new city. The simulation in Task 11 shows that for tier overwrites, 9.98% of historic net sales were attributed to the wrong value. The same distortion would hit regional sales.
Type 2 (new row)	Keeps every version with validity dates.	**Chosen.** Region-wise sales stay correct at the time of the order. Cost: one extra row per change, and 2,000 changes added 2,000 rows (+10%) to a 20,000-row dimension in 0.03 s in the experiment.
Type 3 (previous-value column)	Keeps only current and one previous value.	**Rejected.** Customers who move more than once lose history, and reports by address 'at order time' cannot be answered.

Loyalty tier is initially handled as Type 1, because the original business questions ask only about the current segment. Task 11 shows why this had to be revised when tier history became a requirement.

5.7 Initial load versus incremental load, and load order

	Initial (historical) load	Incremental load (daily; hourly after Task 11)
Extract	Full extract of all history (≥ 24 months).	Only new and changed rows, using an updated_at watermark or the OLTP change log (CDC), taken from a read replica so that billing is not slowed.
Dimensions	Bulk load. Every customer gets a first version with valid_from = first-seen date.	Upsert: compare to the current version and apply SCD (Type 2 for address); insert new members.
Facts	Bulk insert. Indexes and summary tables are created after the load (faster and more compact).	Append new lines only. Re-running a batch must be **idempotent** (the UNIQUE(order_id, line_id) constraint with a batch_id guards against double loading). Cancellations are loaded as reversing rows.
Aggregates	Build from scratch.	Incrementally merge the delta (Task 11).
Run time	Hours; runs once. In my experiment, 1,000,000 lines loaded into the star in 1.28 s plus 1.9 s for indexes and 5.6 s for summaries.	Seconds to minutes.

Load order: (1) dim_date (pre-generated); (2) dim_product; (3) dim_store; (4) dim_promotion; (5) dim_customer (with SCD Type 2); (6) fact_sales; (7) fact_returns (after sales, because it references the sale line); (8) aggregate tables; (9) audit report. Dimensions must precede facts, so that every foreign key can be resolved before a fact is written, and referential integrity is preserved.

5.8 Rejected records and audit of row counts

- Rejected rows are written to `etl_reject` with batch_id, source row, rule ID, reason text and timestamp. They are **never** silently dropped, and the steward corrects and resubmits them.
- **Row-count equation per batch:** rows read = rows loaded + rows rejected (+ duplicates suppressed). A batch with a non-zero difference is flagged as failed and not published to the marts.
- **Control totals:** the sum of the amounts read must equal the amounts loaded plus the amounts rejected.

Control	Value for this extract
Rows read (staging)	6
Rows loaded	4 (1001, 1002, 1004, 1005), of which 1 with an unknown member (1004)
Rows rejected	2 (1003 NEG_QTY, 1006 MISSING_PRODUCT)
Check: read = loaded + rejected	6 = 4 + 2 ✓
Warnings	4 (MATCH_FUZZY, POSSIBLE_DUP, UNKNOWN_STORE, CURRENCY_CONVERTED)
Value loaded (qty × price)	1,240.00 + 620.00 + 382.00 + 2,340.60 = **4,582.60 INR**
Value rejected	−2,340.00 (1003) and 1,250.00 (1006); net −1,090.00
Source value as received after conversion	4,582.60 − 1,090.00 = 3,492.60, which reconciles with loaded plus rejected

Task 6 – Model the Data Cube and Apply OLAP Operations (5 Marks)

QUESTION

Analysts view sales as a cube with dimensions and levels: Time: day → month → quarter → year; Product: product → category → department; Store: store → city → state; Customer segment: segment (no hierarchy). Discuss:

- The total number of cuboids in the lattice with and without the hierarchies (show the calculation)
- The base cuboid and the apex cuboid and what each contains
- One business question for each of roll-up, drill-down, slice, dice and pivot, with the cuboid it uses and an equivalent SQL statement
- Which cuboids you would materialise if storage allows only a small fraction of the lattice, and why

ANSWER

6.1 Number of cuboids

Without hierarchies: each of the n = 4 dimensions is either present or absent (ALL), so the number of cuboids is 2^n = 2^4 = **16**.

With hierarchies: a dimension with L_i levels can appear at any of its L_i levels or at 'ALL', which gives (L_i + 1) choices, so the total is ∏(L_i + 1).

Dimension	Levels (excluding ALL)	L_i	Choices L_i + 1
Time	day, month, quarter, year	4	5
Product	product, category, department	3	4
Store	store, city, state	3	4
Customer segment	segment	1	2

Total = (4+1) × (3+1) × (3+1) × (1+1) = 5 × 4 × 4 × 2 = **160 cuboids**, which is ten times the 16 without hierarchies. The lattice grows multiplicatively with every hierarchy level, so it must never be fully materialised for large dimensions.

6.2 Base and apex cuboids

	Group-by levels	What it contains	Size at FreshCart
Base cuboid	(day, product, store, segment), the lowest level of every dimension	Net sales and quantity for each combination of day, product, store and segment. It is the most detailed cuboid that can be built from the fact data, and every other cuboid is derived from it.	At most min(number of fact rows, 675 days × 5,000 products × 120 stores × 5 segments). In practice, nearly as many rows as the fact table.
Apex cuboid	(ALL, ALL, ALL, ALL)	One row: grand total net sales (and quantity) of FreshCart.	1 row (in the 1M-row test: 248,282,422.18)

6.3 The five OLAP operations tied to FreshCart questions

Cuboid notation: (Time level, Product level, Store level, Customer level). All SQL runs against the Task 3 star schema. A starting view (quarter, category, region) is used for the navigation operations.

Operation	Business question (FreshCart)	Cuboid used	How the cuboid changes
Roll-up	What are the yearly net sales of each department? (Management wants to move from category/quarter detail to the summary view.)	(year, department, ALL, ALL)	from (quarter, category, ALL, ALL): climb Time quarter → year and Product category → department
Drill-down	Region South's sales fell in 2026-Q2. Which states in South are responsible?	(quarter, ALL, state, ALL)	from (quarter, ALL, region, ALL): descend Store region → state
Slice	For one quarter, 2026-Q3, show net sales by category and region.	(ALL, category, region, ALL) after fixing Time = 2026-Q3	selects one value on one dimension; that dimension is removed
Dice	For departments Dept-01 and Dept-02, in Tamil Nadu and Karnataka, for the quarters 2026-Q2 and 2026-Q3, what are the sales by customer segment?	(quarter, department, state, segment) restricted to a sub-cube	selects several values on several dimensions (a sub-cube)
Pivot	Compare category sales across quarters side by side, with categories as rows and quarters as columns.	(quarter, category, ALL, ALL)	same cuboid; the presentation is rotated (quarters become columns)

```
-- 1. ROLL-UP: (quarter, category) -> (year, department)
SELECT p.department_name, d.year_no, SUM(f.net_amount) AS net_sales
FROM fact_sales f JOIN dim_date d ON d.date_key=f.date_key JOIN dim_product p ON
p.product_key=f.product_key
GROUP BY p.department_name, d.year_no;

-- 2. DRILL-DOWN: region -> state (within South)
SELECT d.year_quarter, s.state_name, SUM(f.net_amount) AS net_sales
FROM fact_sales f JOIN dim_date d ON d.date_key=f.date_key JOIN dim_store s ON
s.store_key=f.store_key
WHERE s.region_name='South' AND d.year_quarter IN ('2026-Q1','2026-Q2')
GROUP BY d.year_quarter, s.state_name ORDER BY s.state_name, d.year_quarter;

-- 3. SLICE: Time = '2026-Q3'
SELECT p.category_name, s.region_name, SUM(f.net_amount) AS net_sales
FROM fact_sales f JOIN dim_date d ON d.date_key=f.date_key JOIN dim_product p ON
p.product_key=f.product_key
     JOIN dim_store s ON s.store_key=f.store_key
WHERE d.year_quarter='2026-Q3' GROUP BY p.category_name, s.region_name;

-- 4. DICE: sub-cube on three dimensions
SELECT c.segment, p.department_name, s.state_name, SUM(f.net_amount) AS net_sales
FROM fact_sales f JOIN dim_date d ON d.date_key=f.date_key JOIN dim_product p ON
p.product_key=f.product_key
     JOIN dim_store s ON s.store_key=f.store_key JOIN dim_customer c ON
c.customer_key=f.customer_key
WHERE d.year_quarter IN ('2026-Q2','2026-Q3') AND p.department_name IN ('Dept-01','Dept-02')
  AND s.state_name IN ('Tamil Nadu','Karnataka')
GROUP BY c.segment, p.department_name, s.state_name;

-- 5. PIVOT: categories as rows, quarters as columns
SELECT p.category_name,
  SUM(CASE WHEN d.year_quarter='2026-Q1' THEN f.net_amount END) AS q1_2026,
  SUM(CASE WHEN d.year_quarter='2026-Q2' THEN f.net_amount END) AS q2_2026,
  SUM(CASE WHEN d.year_quarter='2026-Q3' THEN f.net_amount END) AS q3_2026
FROM fact_sales f JOIN dim_date d ON d.date_key=f.date_key JOIN dim_product p ON
p.product_key=f.product_key
GROUP BY p.category_name;
```

Task 7 runs the same operations interactively on the 1,000,000-row warehouse. For example, slicing Store.region = South and dicing the quarters 2026-Q1 and 2026-Q2 returned net sales of **17,115,294.06** and **17,258,080.66**, a rise of about 0.8% quarter on quarter.

6.4 Which cuboids to materialise when only a small fraction of the lattice fits

The method is the greedy benefit-per-cuboid heuristic. The base cuboid is always kept (it is the fact table). Then repeatedly choose the cuboid that most reduces the cost of answering the expected

queries, where cost ≈ rows of the smallest materialised cuboid that can answer them. A cuboid is useful only if it is much smaller than its cheapest materialised ancestor. For example, (day, department, store) has up to 675 × 12 × 120 ≈ 972,000 rows, almost as large as the base, so it gives almost no benefit and is skipped.

Materialise	Max rows	Questions it answers (and everything above it in the lattice)
(quarter, category, region, ALL)	8 × 60 × 3 = 1,440 (measured: 1,440)	Q1, Q9 and the roll-up, slice, pivot examples. Also any coarser cuboid such as (year, department).
(month, category, state, segment)	22 × 60 × 6 × 5 = 39,600	Q6, Q4 (segment) and drill-down to state or month; the **workhorse** cuboid for dashboards. All roll-ups to quarter/year, department, region, or ALL come from it.
(day, ALL, store, ALL)	675 × 120 = 81,000	Q5 (year-to-date vs same period last year by store) and Q7 (discount by store and month).
(day-type, department, ALL, ALL) → or (day, department, state)	24 / 48,600	Q3 (weekday vs weekend by department). The day-type version has only 24 rows (measured: 24).
(promotion, store) + distinct-order counter	26 × 120 = 3,120 (measured: 3,120)	Q2 (average basket value, which needs an order count that cannot be re-derived from other cuboids).

These five cuboids hold about 125,000–175,000 rows in total against 1,000,000+ rows in the fact table, and are 5 of 160 (3%) of the lattice. They cover the ten business questions of Task 1. Selection criteria: (1) closeness to the actual questions, (2) size: each is a small fraction of the base, (3) coverage: cuboids at the finest level of the common queries serve all their ancestors, and (4) refresh cost, because every materialised cuboid must be refreshed on every load (Task 11 shows incremental refresh).

Task 7 – Develop an OLAP Application (5 Marks)

QUESTION

Develop an application that performs OLAP operations on your warehouse (SQL with Python, or Pentaho/Mondrian, Apache Superset or Metabase). Ensure that:

- The warehouse is populated with realistic data of about 1,000 → 10,000 → 100,000 → 1,000,000 fact rows, with a documented data-generation method
- The user chooses the operation, dimension, level and filter values through input, rather than running fixed queries
- Roll-up, drill-down, slice, dice and pivot are all demonstrated
- Results are cross-checked against an independent calculation

ANSWER

7.1 Approach and files

The application is a Python console program (`olap_app.py`, SQLite via the standard library, pandas for display and pivots) on the star schema of Task 3. It accepts commands typed by the user, and builds the SQL from the user's choices, so no query is hard-coded. The data generator and the ETL that builds the star from the OLTP-style database is `freshcart_dw.py`. Both are listed in the appendices and provided as files.

7.2 Data-generation method (documented)

- **Tool and seed:** Python + NumPy with seed 42 (reproducible). Command: `python freshcart_dw.py build N` for N = 1,000; 10,000; 100,000; 1,000,000.
- **Reference data:** 3 regions, 6 states, 70 cities, 120 stores; 12 departments, 60 categories, 200 brands, 5,000 products (log-normal list prices with a median of about ₹110); 20,000 customers in five segments (Premium 12%, Regular 45%, Budget 25%, New 12%, Lapsed 6%); 25 promotions with 5–25% discounts. (The 50,000-product figure in Task 4 is used for the storage calculation only.)
- **Orders and lines:** 1 to 5 lines per order (average about 3.3); orders that carry a promotion (30%) get one extra line so promotions raise basket size; line quantity is geometric (1–8). Order dates span 2025-01-01 to 2026-11-05, with weekend days weighted 1.4× so that weekend and weekday patterns differ.
- **Returns:** about 4% of lines are returned 1–14 days after sale, for 1–2 units.
- **Normalised first, then star:** data are first written to a normalised 13-table OLTP-style database, and the star is then built from it with the ETL of Task 5. That makes independent reconciliation possible.

Size (lines)	Orders	OLTP size (MB)	Star size (MB)	Generation time (s)	Star ETL time (s)
1,000	308	1.1	1.9	0.07	0.02
10,000	2,996	1.4	2.4	0.09	0.04
100,000	30,434	4.8	7.2	0.30	0.16
1,000,000	303,402	39.4	57.2	2.24	1.30

7.3 User interaction

Command	Effect	OLAP operation
`rollup <dim>`	Moves the dimension one level up its hierarchy (final level is ALL)	Roll-up
`drilldown <dim>`	Moves the dimension one level down	Drill-down
`slice <dim> <level> <value>`	Fixes one member (e.g. `slice store region South`) and drops the dimension from the grouping	Slice
`dice <dim> <level> v1,v2,…`	Restricts to several members, keeping the grouping	Dice
`pivot <rowdim> <coldim>`	Shows a cross-tab with totals	Pivot
`measure net|qty|gross`, `reset`, `sql`, `reconcile`	Choose measure, restore the view, show the generated SQL, run reconciliation	—

7.4 Demonstration on 1,000,000 fact rows (console capture)

The captured session below was produced by the application with 1,000,000 lines. Rows have been shortened where marked '…'. GUI screenshots can be taken by running `python olap_app.py --size 1000000` and typing the same commands; the text below is the console output of that run.

```
FreshCart OLAP - star schema with 1,000,000 sales lines. Type 'help'.
olap> reset
reset to default view
olap> rollup product
roll-up product: now at level 'department'
   year_quarter department_name region_name         net
0       2025-Q1         Dept-01       North   471557.75
1       2025-Q1         Dept-01       South  1443416.57
2       2025-Q1         Dept-01        West   904507.87
3       2025-Q1         Dept-02       North   461457.53
4       2025-Q1         Dept-02       South  1503570.80
5       2025-Q1         Dept-02        West   917622.99
6       2025-Q1         Dept-03       North   399729.08
7       2025-Q1         Dept-03       South  1369863.47
   …
olap> rollup product
roll-up product: now at level 'ALL'
   year_quarter region_name          net
0       2025-Q1       North   5168917.90
1       2025-Q1       South  17082026.50
2       2025-Q1        West  10889955.72
3       2025-Q2       North   5404728.87
4       2025-Q2       South  17242611.25
5       2025-Q2        West  10753423.22
6       2025-Q3       North   5476461.50
7       2025-Q3       South  17426356.89
   …
olap> rollup store
roll-up store: now at level 'ALL'
  year_quarter          net
0      2025-Q1  33140900.12
1      2025-Q2  33400763.34
2      2025-Q3  33790125.99
3      2025-Q4  34091311.80
4      2026-Q1  33171495.86
5      2026-Q2  33442218.35
6      2026-Q3  33897843.57
7      2026-Q4  13347763.15
olap> drilldown store
drill-down store: now at level 'region'
   year_quarter region_name          net
0       2025-Q1       North   5168917.90
1       2025-Q1       South  17082026.50
2       2025-Q1        West  10889955.72
3       2025-Q2       North   5404728.87
4       2025-Q2       South  17242611.25
5       2025-Q2        West  10753423.22
```

```
6       2025-Q3       North   5476461.50
7       2025-Q3       South  17426356.89
   …
olap> dice time quarter 2026-Q1,2026-Q2
dice time.quarter IN ['2026-Q1', '2026-Q2']
  year_quarter region_name          net
0      2026-Q1       North   5252214.38
1      2026-Q1       South  17115294.06
2      2026-Q1        West  10803987.42
3      2026-Q2       North   5252318.83
4      2026-Q2       South  17258080.66
5      2026-Q2        West  10931818.86
olap> slice store region South
slice store.region = South (dimension removed from grouping)
  year_quarter          net
0      2026-Q1  17115294.06
1      2026-Q2  17258080.66
olap> sql
SELECT d.year_quarter, ROUND(SUM(f.net_amount),2) AS net FROM fact_sales f JOIN dim_date d ON
d.date_key=f.date_key JOIN dim_store s ON s.store_key=f.store_key WHERE d.year_quarter IN ('2026
olap> drilldown product
drill-down product: now at level 'department'
   year_quarter department_name         net
0       2026-Q1         Dept-01  1467489.92
1       2026-Q1         Dept-02  1510397.79
2       2026-Q1         Dept-03  1351253.97
3       2026-Q1         Dept-04  1537442.97
4       2026-Q1         Dept-05  1482910.77
5       2026-Q1         Dept-06  1521864.72
6       2026-Q1         Dept-07  1419830.47
7       2026-Q1         Dept-08  1434064.29
   …
olap> pivot product time
pivot rows=product columns=time
year_quarter         2026-Q1      2026-Q2        TOTAL
department_name
Dept-01           1467489.92   1464263.14   2931753.06
Dept-02           1510397.79   1512256.08   3022653.87
Dept-03           1351253.97   1314563.86   2665817.83
Dept-04           1537442.97   1543177.37   3080620.34
Dept-05           1482910.77   1533598.45   3016509.22
Dept-06           1521864.72   1517179.95   3039044.67
Dept-07           1419830.47   1401460.53   2821291.00
Dept-08           1434064.29   1489498.54   2923562.83
Dept-09           1252753.78   1334408.82   2587162.60
Dept-10           1340410.23   1336542.64   2676952.87
Dept-11           1441630.36   1493939.55   2935569.91
Dept-12           1355244.79   1317191.73   2672436.52
TOTAL            17115294.06  17258080.66  34373374.72
olap> sql
SELECT d.year_quarter, p.department_name, ROUND(SUM(f.net_amount),2) AS net FROM fact_sales f JOIN
dim_date d ON d.date_key=f.date_key JOIN dim_product p ON p.product_key=f.product_key JOIN
olap> reconcile
view total 34,373,374.72  vs  same filters at apex cuboid 34,373,374.72  -> OK
unfiltered net sales: star 248,282,422.18 (1000000 rows) | OLTP SQL 248,282,422.18 (1000000 rows) |
pandas 248,282,422.18
row counts and grand totals reconcile -> OK
```

7.5 Reconciliation against independent calculations

The `reconcile` command performs two checks. (1) **Internal consistency:** the total of the current view equals the apex cuboid computed with the same filters (roll-up must not change totals). (2) **Independent calculation from the source:** the grand total of net sales and the row count in the star are compared with (a) a separate SQL calculation on the normalised OLTP tables, using SUM(qty × unit_price − discount) and (b) a pandas calculation over the raw order-line rows. The three results must agree to the paisa.

Size (lines)	Star SUM(net)	OLTP SQL SUM	Summary table SUM	Rows star / OLTP	Result
1,000	246,374.57	246,374.57	246,374.57	1,000 / 1,000	Reconciled ✓

Size (lines)	Star SUM(net)	OLTP SQL SUM	Summary table SUM	Rows star / OLTP	Result
10,000	2,484,263.54	2,484,263.54	2,484,263.54	10,000 / 10,000	Reconciled ✓
100,000	24,697,522.28	24,697,522.28	24,697,522.28	100,000 / 100,000	Reconciled ✓
1,000,000	248,282,422.18	248,282,422.18	248,282,422.18	1,000,000 / 1,000,000	Reconciled ✓

The star schema therefore carries the complete content of the source: no rows were lost or duplicated, and the grand total reproduces the source figure at every size.

Task 8 – Evaluate Query Processing Performance (4 Marks)

QUESTION

Select three representative analytical queries. Execute each on (i) the normalised OLTP-style schema, (ii) your star schema, (iii) your star schema with indexes and/or a summary table. Record for each data size: execution time; rows scanned or read; number of joins and the query plan; storage and load time of the additional structures. Explain why the differences occur and what trade-off the summary structures introduce. Execution time must not be the only evidence.

ANSWER

8.1 Method

- **Engine and environment:** SQLite 3.45 on a single CPU core in a sandbox; the same engine for every variant, so differences come from the schema and structures, not the engine. Cache is warm; each time is the best of 3 runs.
- **Queries:** Q1 net sales by region × category × quarter (the Task 1 question 1); Q2 average basket value by promotion × state; Q3 weekend vs weekday net sales by department.
- **Variants:** (i) normalised OLTP, with and without the four indexes I would add (shown as (i-b)); (ii) star, no extra structures; (iii-a) star + 6 foreign-key indexes; (iii-b) star + indexes + three summary tables (quarter × region × category; promotion × store with order counts; day-type × department).
- **Evidence beyond time:** the number of joins, the query plan, the full-scan rows read (rows of tables read by sequential SCAN steps; probes by primary key or index are counted separately), and the VM-instruction count (in thousands, from a progress-handler counter in the engine), which does not depend on machine noise.
- **Limitations:** SQLite is a row-store with no parallelism or columnar storage, so absolute times differ from a real warehouse engine. The relative effects are what matter, and are reported as such.

8.2 Execution time (milliseconds, best of 3)

Query	Variant	1K	10K	100K	1M
Q1 Net sales by region x category x quarter	(i) OLTP 3NF	2.1	24.1	199	1981
	(i-b) OLTP + indexes	2.2	21.8	250	13866
	(ii) Star	1.5	14.8	141	1368
	(iii-a) Star + indexes	1.8	19.8	218	2372
	(iii-b) Star + summary table	0.6	2.1	1.4	1.3
Q2 Avg basket value by promotion x state	(i) OLTP 3NF	0.6	7.6	64.8	721
	(i-b) OLTP + indexes	0.6	10.2	66.2	608
	(ii) Star	0.5	6.3	56.3	586
	(iii-a) Star + indexes	0.4	4.7	41.5	407
	(iii-b) Star + summary table	0.2	1.2	1.7	1.6
Q3 Weekend vs weekday net sales by department	(i) OLTP 3NF	1.1	14.5	150	1611

Query	Variant	1K	10K	100K	1M
	(i-b) OLTP + indexes	1.2	27.5	201	2942
	(ii) Star	0.7	8.6	94.3	965
	(iii-a) Star + indexes	0.7	11.8	161	1956
	(iii-b) Star + summary table	<0.1	<0.1	<0.1	<0.1

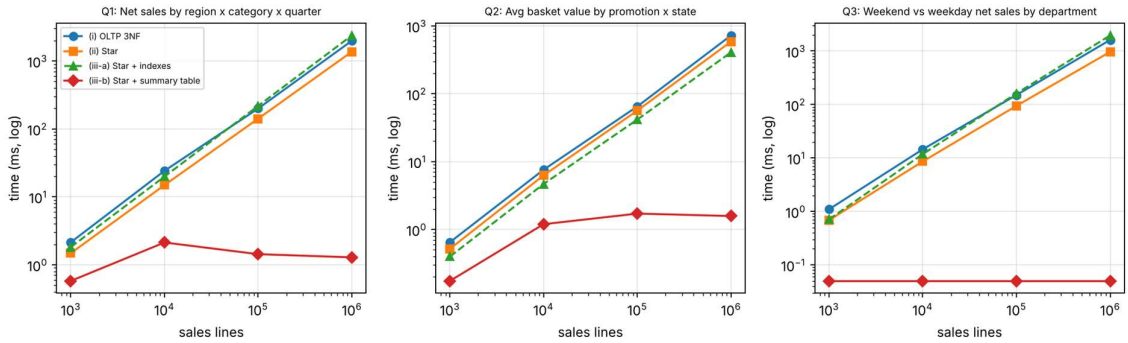


Figure 8.1 – Query time versus data size (log–log) for the three queries and four main variants

8.3 Work done at 1,000,000 lines (joins, rows scanned, engine work)

Query	Variant	Joins	Rows full-scanned	Index/PK probe steps	Engine work (k VM ops)	Result rows
Q1	(i) OLTP 3NF	8	1,000,000	8	53,037	1440
	(i-b) OLTP + indexes	8	206	7	108,198	1440
	(ii) Star	3	1,000,000	3	26,024	1440
	(iii-a) Star + indexes	3	5,000	3	26,040	1440
	(iii-b) Star + summary table	0	1,440	0	10	1440
Q2	(i) OLTP 3NF	5	1,000,000	5	40,442	156
	(i-b) OLTP + indexes	5	303,402	5	29,352	156
	(ii) Star	2	1,000,000	2	29,228	156
	(iii-a) Star + indexes	2	1,000,000	2	21,228	156
	(iii-b) Star + summary table	2	3,120	2	75	156
Q3	(i) OLTP 3NF	5	1,000,000	5	42,180	24
	(i-b) OLTP + indexes	5	200	5	36,232	24
	(ii) Star	2	1,000,000	2	22,001	24
	(iii-a) Star + indexes	2	5,000	2	22,015	24
	(iii-b) Star + summary table	0	24	0	0	24

Notes: for (i) and (ii) the full scan reads all 1,000,000 fact/OrderLines rows and then probes dimensions by primary key, once per row per join; “rows full-scanned” counts only sequential scans, so the probe steps column shows how many lookup stages the plan has. For (iii-b), the engine reads only the 24–3,120 rows of the summary table.

8.4 Query plans at 1,000,000 lines

```
Q1 (i) OLTP – 8 joins:
  SCAN l
  SEARCH o USING INTEGER PRIMARY KEY (rowid=?)
  SEARCH p USING INTEGER PRIMARY KEY (rowid=?)
  SEARCH b USING INTEGER PRIMARY KEY (rowid=?)
  SEARCH c USING INTEGER PRIMARY KEY (rowid=?)
  SEARCH s USING INTEGER PRIMARY KEY (rowid=?)
  SEARCH ci USING INTEGER PRIMARY KEY (rowid=?)
  SEARCH st USING INTEGER PRIMARY KEY (rowid=?)
  SEARCH r USING INTEGER PRIMARY KEY (rowid=?)
  USE TEMP B-TREE FOR GROUP BY

Q1 (ii) Star – 3 joins:
  SCAN f
  SEARCH d USING INTEGER PRIMARY KEY (rowid=?)
  SEARCH p USING INTEGER PRIMARY KEY (rowid=?)
  SEARCH s USING INTEGER PRIMARY KEY (rowid=?)
  USE TEMP B-TREE FOR GROUP BY

Q1 (iii-a) Star + indexes – planner switches to index nested loop on product_key:
  SCAN p
  SEARCH f USING INDEX ix_f_prod (product_key=?)
  SEARCH d USING INTEGER PRIMARY KEY (rowid=?)
  SEARCH s USING INTEGER PRIMARY KEY (rowid=?)
  USE TEMP B-TREE FOR GROUP BY

Q1 (iii-b) Star + summary table – 0 joins:
  SCAN agg_qtr_region_cat
```

```
Q2 (ii) Star:              CO-ROUTINE t | SCAN fact_sales | USE TEMP B-TREE FOR GROUP BY | SCAN t | SEARCH p USING INTEGER PRIMARY KEY (rowid=?) | SEARCH s USING INTEGER PRIMARY KEY (rowid=?) | USE TEMP B-TREE FOR GROUP BY

Q2 (iii-a) Star + indexes:  CO-ROUTINE t | SCAN fact_sales USING INDEX ix_f_order | SCAN t | SEARCH s USING INTEGER PRIMARY KEY (rowid=?) | SEARCH p USING INTEGER PRIMARY KEY (rowid=?) | USE TEMP B-TREE FOR GROUP BY

Q3 (ii) Star:               SCAN f | SEARCH dd USING INTEGER PRIMARY KEY (rowid=?) | SEARCH p USING INTEGER PRIMARY KEY (rowid=?) | USE TEMP B-TREE FOR GROUP BY

Q3 (iii-b) Summary:         SCAN agg_day_type_dept
```

8.5 Storage and load time of the additional structures

Size (lines)	OLTP (MB)	Star base (MB)	Star ETL load (s)	Indexes: extra MB / build (s)	Summary tables: extra MB / rows / build (s)	OLTP indexes: extra MB / build (s)
1,000	1.1	1.9	0.02	0.10 / 0.01	0.04 / 896 / 0.01	0.05 / 0.01
10,000	1.4	2.4	0.04	0.63 / 0.03	0.09 / 2,309 / 0.05	0.31 / 0.02
100,000	4.8	7.2	0.16	6.39 / 0.19	0.14 / 4,435 / 0.47	2.98 / 0.10
1,000,000	39.4	57.2	1.30	66.35 / 1.90	0.14 / 4,584 / 5.65	31.71 / 0.99

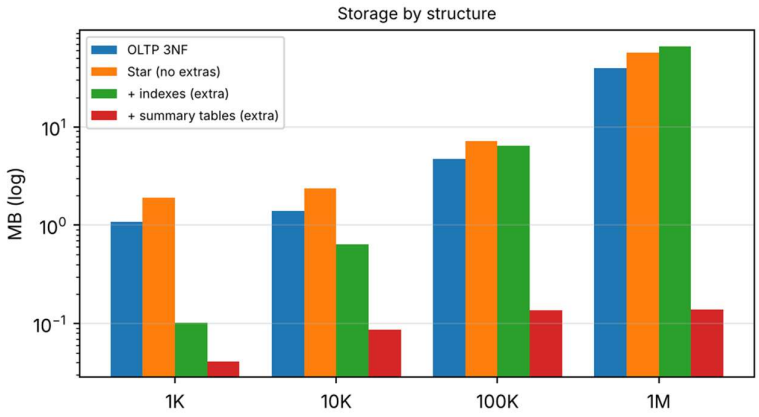


Figure 8.2 – Storage by structure

8.6 Interpretation: why the differences occur

- **Star vs OLTP (about 1.2–1.7× faster at 1M).** All variants must read every sales line once, so the dominant cost is the same scan. The difference is the number of lookup stages per row: Q1 does 8 primary-key probes per OLTP row against 3 on the star, and the engine-work counter falls from 53,037k to 26,024k operations (2.0× fewer). The denormalised dimension also saves the brand → category chain, and Orders is not needed because date and store are keys in the fact. Time scales linearly with data (1,000 → 1,000,000 rows is about 1,000× more data and 700–1,000× more time), which is the real problem: **a scan-and-join design cannot stay interactive as FreshCart grows.**
- **Indexes alone did not help full-aggregation queries, and made two of them slower.** With six foreign-key indexes the planner chose to walk the Product table and fetch each product's fact rows through ix_f_prod. For Q1 this was 1.7× slower (2.37 s vs 1.37 s) and for Q3 2.0× slower, because these queries read 100% of the table and index look-ups turn a sequential scan into 5,000 batches of scattered reads. The full-scan row count dropped from 1,000,000 to 5,000, which shows why 'rows scanned' alone can mislead: the probes cost more than the scan they replace. Only Q2 gained (31% faster), because the order_id index delivers rows already grouped by order and removes a sort. The indexes also cost **66 MB (+116% of the star)** and 1.9 s of build time, and must be maintained on every load. Indexes help selective queries (drill-through to one order, dimension lookups in ETL), not whole-table aggregation. The same happened on the OLTP schema (Q1: 13.9 s with indexes vs 2.0 s without), so this is a property of the access pattern.
- **Summary tables changed the cost model.** Q1 falls from 1368 ms to 1.3 ms (1,066× faster), Q2 from 586 ms to 1.6 ms (372×), and Q3 from 965 ms to under 0.1 ms. The work depends on the **size of the summary** (24 to 3,120 rows), not on the number of fact rows, so the response time stays flat at 1K, 10K, 100K and 1M (see the flat line in Figure 8.1). The summary tables cost only 0.14 MB, but 5.6 s to build from the whole fact table.
- **The trade-off the summary structures introduce:** (1) **Refresh cost:** every load makes the summaries stale; rebuilding them is a full scan (5.7 s at 1M rows, growing linearly), so for hourly refresh they must be maintained incrementally (Task 11). (2) **Flexibility:** a summary answers only questions at its grain or coarser. A question at a finer level (e.g. by day and product) falls back to the fact table. (3) **Non-additive measures:** averages and distinct counts are stored as their additive components (net sales and order count), otherwise rolling up gives wrong answers. (4) **Consistency risk:** summaries must reconcile with the facts, and the

reconciliation in Task 7 shows that they do. (5) **Storage/maintenance growth** with the number of cuboids (Task 6), which is why only a few are materialised.

Summary of Task 8: the star schema is a modest improvement (1.2–1.7×); indexes are a poor investment for whole-table aggregates and a real cost (+116% storage); summary tables give 300–1,400× faster answers for the cost of a small table plus a managed refresh. The recommended structure is therefore star + a few well-chosen summary tables, with indexes only where selective access is required.

Task 9 – Build a KNIME Reporting Workflow (3 Marks)

QUESTION

Build a KNIME workflow that reads warehouse data (CSV or database), cleans it, aggregates it and produces a management report. Discuss:

- The nodes used (File/DB Reader, Joiner, GroupBy, Pivoting, Row Filter, charts) and the purpose of each
- How each part of the workflow corresponds to ETL or OLAP concepts from Tasks 5 and 6
- One limitation of the tool when the data grows to 1,000,000+ rows

ANSWER

Please note: KNIME Analytics Platform is a desktop application and cannot be run in this environment, so I have not produced a real KNIME screenshot or a `.knwf` file. What is provided is (a) the input files exported from the 100,000-row warehouse (`fact_sales_100k.csv`, `dim_date_100k.csv`, `dim_product_100k.csv`, `dim_store_100k.csv`), (b) a node-by-node build specification below, which takes about 15 minutes to assemble, and (c) the expected report figures, computed independently with pandas so that you can check your KNIME output against them. For submission, please build the workflow from this specification and take your own annotated screenshot (File → Export and Image to Report).

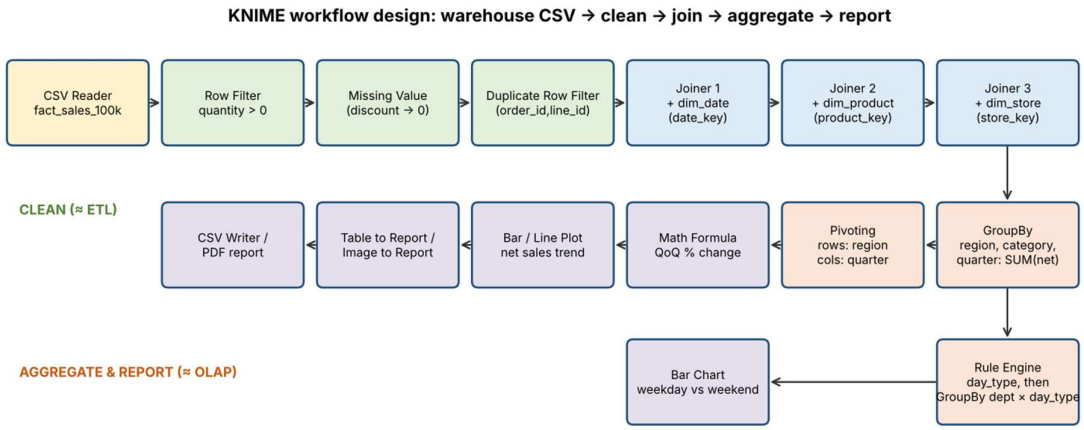


Figure 9.1 – KNIME workflow design (schematic of the nodes to be built; not a screenshot)

9.1 Nodes, configuration and purpose

#	Node	Configuration	Purpose
1	**CSV Reader** (×4)	fact_sales_100k.csv, dim_date_100k.csv, dim_product_100k.csv, dim_store_100k.csv. Comma separator, first row = header.	Reads the star-schema tables (fact table and conformed dimensions) exported from the warehouse.
2	**Row Filter**	Include rows where quantity > 0 and net_amount ≥ 0.	Removes invalid sales lines (the NEG_QTY rule R5 of Task 5).
3	**Missing Value**	discount_amount: replace missing with 0; string columns: replace with 'Unknown'.	Handles NULLs and sets a default (Unknown member, rule R7 of Task 5).
4	**Duplicate Row Filter**	Key columns: order_id and line_id; keep first.	Enforces the declared grain, one row per order line (UNIQUE(order_id, line_id)).
5	**Joiner** (×3)	Inner join on date_key, then product_key, then store_key (left input = fact).	Re-creates the star join: fact + dim_date + dim_product + dim_store.
6	**Rule Engine** (optional)	`$day_type$ = "Weekend" => "WE"` (or use the day_type column directly).	Derives or relabels a dimension attribute.

#	Node	Configuration	Purpose
7	**GroupBy**	Group columns: region_name, category_name, year_quarter. Aggregation: Sum(net_amount), Sum(quantity).	Aggregates the fact to the (quarter, category, region) cuboid.
8	**Pivoting**	Pivot: year_quarter. Group: region_name. Aggregation: Sum(Sum(net_amount)).	Cross-tabulation: regions as rows, quarters as columns.
9	**Math Formula**	(Q − Q_prev) / Q_prev × 100 per column pair.	Computes quarter-over-quarter growth (business question Q1).
10	**GroupBy** (2nd branch)	Group: department_name, day_type; Sum(net_amount).	Weekday vs weekend by department (Q3).
11	**Bar Chart / Line Plot**	X: year_quarter; Y: net sales; series: region. Second chart: department × day type.	Visual management report.
12	**Table to Report / Image to Report, then CSV Writer**	Insert the chart and table into a report, export it to PDF or HTML, and write the aggregate to report_output.csv.	Produces the exported management report.

9.2 Mapping to ETL and OLAP concepts

Workflow part	Concept (Task)	Explanation
CSV Reader	Extract (Task 5)	Reads a snapshot of warehouse tables; in production this would be a database connector with a pushed-down query.
Row Filter, Missing Value, Duplicate Row Filter	Transform / clean (Task 5)	Same rules as R5 (negative quantity), R7 (unknown member) and R10 (duplicate protection).
Joiner ×3	Star join (Tasks 3, 4)	The three joins of the 3-join star query for Q1. A snowflake would need 8.
GroupBy	Roll-up to a cuboid (Task 6)	Grouping by (quarter, category, region) computes that cuboid. Removing a grouping column would roll up further; adding one drills down.
Row Filter on a value (e.g. region = South)	Slice / dice (Task 6)	One value on one dimension is a slice; a set of values is a dice.
Pivoting	Pivot (Task 6)	Rotates quarter from rows to columns, as in 6.3 operation 5.
Math Formula	Derived measure (Task 1)	QoQ growth is a non-additive derived measure, calculated *after* aggregation.
Charts, report, CSV Writer	Front-end / load to presentation (Task 2)	The front-end tool layer of the architecture.

9.3 Expected report content (independently computed with pandas on the same 100,000 lines)

Net sales by region and quarter (₹, rounded). The 2026-Q4 column is partial, because the data stop on 5 November 2026.

Region	2025-Q1	2025-Q2	2025-Q3	2025-Q4	2026-Q1	2026-Q2	2026-Q3	2026-Q4 (part)
North	546,086	596,345	518,039	529,493	554,361	547,734	532,126	181,606
South	1,655,192	1,765,773	1,718,471	1,707,394	1,656,155	1,718,451	1,738,866	690,279
West	1,089,101	1,125,973	1,108,071	1,163,035	985,779	1,106,776	1,057,359	405,056

- **Grand total:** 24,697,522.28 (equals the star and OLTP totals in Task 7).

- **Weekend vs weekday:** weekend days account for 30.4% of net sales (7.52M of 24.70M) with 2 of 7 days (28.6% of days), so weekend days are slightly busier.
- **Largest departments:** Dept-05 (2.22M), Dept-04 (2.16M), Dept-11 (2.15M).
- **Example insight (Q1):** West fell 11.0% from 2026-Q1 vs 2025-Q4 (985,779 vs 1,163,035) and recovered 12.3% in 2026-Q2. North and South moved less than ±3% in the same quarters.

The KNIME GroupBy and Pivoting output must match these figures exactly; any mismatch indicates a cleaning or join error in the workflow.

9.4 Limitation at 1,000,000+ rows

KNIME processes tables in the memory and disk cache of one desktop JVM (heap by default about a quarter of RAM), and every node materialises a full intermediate table. The CSV Reader parses text every time it executes, and the Joiner and GroupBy nodes hold large hash tables. This works for 100,000 rows (a few seconds), but with 1,000,000 fact rows × 20 columns × several intermediate copies it slows, and at 100 million lines the workflow would run out of heap or take unacceptably long, with no pre-aggregation or indexes as the warehouse has. The usual remedies are to **push aggregation into the database** (DB Reader/DB GroupBy nodes so that only the 1,440-row summary reaches KNIME), use the columnar table back end, or read the pre-built summary tables of Task 6 instead of the raw fact.

Task 10 – Recommend and Defend the Final Design (3 Marks)

QUESTION

Recommend the final warehouse design for FreshCart. Discuss: the schema and architecture and why they are better than the alternatives studied; the evidence from Tasks 4 and 8; trade-offs, load cost and limitations; whether the design remains practical at 1,000,000+ rows and beyond. The recommendation must be based on evidence obtained from your own design and experiments.

ANSWER

10.1 Recommended design

Layer	Recommendation
Architecture	Enterprise data warehouse on a server separate from billing, with dependent data marts (Sales; Returns/Customer) and a ROLAP server. Extraction is staged, with ETL from a replica or change log.
Schema	**Star schema** with conformed dimensions (Date, Product, Store, Customer, Promotion) and two fact tables (Sales at order-line grain, Returns at returned-line grain). Customer address as SCD Type 2.
Pre-computation	A **small set of summary tables** chosen from the cuboid lattice (Task 6.4), maintained incrementally. **Indexes only for selective access** (surrogate-key lookups in ETL, the order_id index for drill-through), not blanket foreign-key indexes on the fact.
Governance	Single net-sales definition in the metadata repository; every load audited by row count and control total.

10.2 Evidence from my own experiments

Evidence	Result (1,000,000 lines)	Decision it supports
Task 4: storage	Snowflake saves 4.02 MB of 7.46 MB of dimension storage (53.9%), but that is only about 6.3% of a 1M-row fact table (64 MB) and 0.06% at 100M rows.	Space is not a reason to snowflake.
Task 4: joins	Q1 needs 3 joins on the star but 8 on a snowflake (+5); 2 vs 4 and 2 vs 5 for Q2 and Q3.	Star is simpler for users and faster.
Task 8: star vs normalised	Q1 1.98 s → 1.37 s (1.45×); Q2 1.23×; Q3 1.67×. Engine work down 2.0× (Q1).	Star gives a consistent, moderate gain; the main benefit is simplicity and stable semantics.
Task 8: indexes	+66 MB (+116%), 1.9 s build; Q1 1.7× and Q3 2.0× slower; Q2 31% faster.	Do not index every foreign key; keep only indexes with a measured benefit.
Task 8: summary tables	Q1 1,066×, Q2 372× faster; Q3 down to under 0.1 ms. Cost 0.14 MB and 5.6 s build; flat response time from 1K to 1M.	Summary tables make interactive analysis independent of fact size.
Task 7: reconciliation	Star, OLTP, pandas and summary totals agree at all four sizes (248,282,422.18 at 1M).	The dimensional model is complete and correct.

10.3 Trade-offs, load cost and limitations

- **Redundancy and update cost:** denormalised dimensions repeat text (a brand reclassification touches about 62 product rows), and Type 2 history adds rows. This is acceptable because hierarchies change rarely.
- **Load cost:** initial star load for 1M lines took 1.3 s, indexes 1.9 s, summaries 5.6 s. Every summary table adds refresh work to every load, which is why only five are selected.
- **Flexibility trade-off:** a summary answers only questions at or above its grain. Novel questions run against the fact table, more slowly.

- **Limitations of my evidence:** SQLite on one core; synthetic data with uniform distributions (no real skew, such as a few best-selling products); no concurrent users; warm cache; ten-minute-scale effects of real hardware are not captured. Relative comparisons are reliable, but absolute times are not predictions for production.

10.4 Is it practical at 1,000,000+ rows and beyond?

Yes for up to roughly 10^8 rows with the same design; beyond that, with changes in the engine rather than in the model. Scan-based queries scale linearly: Q1 on the star takes 1.37 s at 1M, so about 137 s at 100M and about 12 minutes at the estimated 525M lines per year. This is not interactive without summary tables, which stay at about 1 ms because they depend on the number of cuboid cells (1,440 rows), not on the number of facts. At and above 10^8 rows, I would keep the same star schema and cuboid selection but move to: monthly partitioning of fact_sales (so that a refresh and a query touch only the relevant months), a columnar or MPP engine, and incremental maintenance of the summaries (Task 11). The dimensional model (grain, conformed dimensions, SCD) stays unchanged, which is the main advantage of designing the model separately from the physical structures.

Task 11 – Re-evaluate and Adapt the Design (8 Marks)

QUESTION (Day-4 Design Challenge)

FreshCart launches an online channel. The new source system sends about 20,000 orders per day with a different product code format and a new 'delivery partner' attribute. Customers change address and loyalty tier frequently, and the business must report sales by the address and tier that applied at the time of the order. Executives want dashboards refreshed every hour. Re-evaluate the original design: retain, modify or redesign.

A. Impact analysis (2 marks) B. Evaluation of the original design (2 marks) C. Adaptation or redesign (2 marks) D. Evidence-based defence (2 marks)

ANSWER

Decision in one line: keep the star schema, the grain and the conformed dimensions (retain), modify the schema additively (new dimensions, new fact keys, SCD changes), and redesign the load and aggregate refresh from nightly full rebuild to hourly micro-batch with incremental aggregates.

A. Impact analysis

Online volume is about 20,000 orders per day. At an average of about 4 lines per order, that is about 80,000 lines per day and 29 million per year, added to the store data. In my simulation, one day of online orders was 80,296 lines.

Component	Affected?	Kind of change	What changes
Grain of fact_sales (one row per order line)	**No**	—	An online order line has the same grain, so both channels sit in one fact table.
Measures (quantity, gross, discount, net)	**No**	—	Same definitions, so 'net sales' stays consistent across channels.
dim_date, dim_promotion	No	—	Unchanged (for hourly analysis, an optional hour-of-day attribute can be added).
dim_product	**Yes**	Structure + load	New product code format. Add xref_product_code (source system, source code → product_key). The surrogate-key design of Task 3 is what makes this an additive change.
dim_store	**Yes**	Structure	An 'ONLINE' fulfilment member (key 121) because an online order has no store.
New dim_channel	**Yes**	Structure	Store / Online, so that every report can split by channel.
New dim_delivery_partner	**Yes**	Structure	New attribute, with a 'Not applicable' member (0) for in-store sales.
dim_customer	**Yes**	Structure + load	Address **and** tier must now be correct at order time, which means SCD Type 2 for both (tier was Type 1 originally).
fact_sales	**Yes**	Structure	Add channel_key, delivery_partner_key (and loyalty_tier_key, see C). Existing rows get defaults (Store, Not applicable).
ETL process	**Yes**	Load	Second extract feed; product-code crosswalk; currency/format rules for the new source; hourly micro-batch instead of nightly batch; idempotent loads.
Summary tables / cubes	**Yes**	Performance	Full rebuild (a scan of all facts) cannot run each hour as data grows; refresh must be incremental. Existing cuboids remain valid, with channel as an additional grouping where needed.
Architecture	**Mostly no**	Load / performance	The staging area gets a second source and runs at a shorter interval. The enterprise warehouse + marts + ROLAP structure is unchanged.

B. Evaluation of the original design against the new requirement

I tested the original design on a copy of the 1,000,000-line warehouse, by (1) simulating one hour of online orders (833 orders, 3,375 lines) and one day (20,000 orders, 80,296 lines), (2) simulating 2,000 customers changing address and tier, and (3) timing the aggregate refresh.

Test	Result	Verdict for the original design
Product-code lookup of online lines against the original dim_product (match on SKU)	**0.0% matched**, so all 3,375 lines would fall to Unknown/reject	**Fails.** Structure is adequate (surrogate keys), but there is no mapping from the new codes.
Delivery partner	No column or dimension to hold it	**Fails** (structure).
Report 'by tier at the time of the order' with tier as SCD Type 1	Overwriting the tier of 2,000 of 20,000 customers (10%) misattributes **9.98% of historic net sales**	**Fails** for tier. Address is already Type 2, so it **passes**.
SCD Type 2 cost for 2,000 changes	0.028 s; dimension grows from 20,000 to 22,000 rows (+10%)	**Adequate.** Cheap.
Load one hour of online lines (indexes in place)	0.15 s for 3,375 lines	**Adequate.** Far inside the 1-hour window.
Load one full day of online lines	0.88 s for 80,296 lines	**Adequate.**
Hourly refresh by **full rebuild** of the three summary tables	5.58 s at 1.0M rows (≈ 645M rows per hour of rebuild capacity). Linear: ≈ 163 s at 29M rows (one year of online lines) and ≈ 49 min at the 525M-line estimate	**Fails at production scale.** It fits a nightly window but not an hourly one once the fact table passes about 600 million rows. It also repeats all the work on data that did not change.
Incremental refresh of the summaries (merge one hour's delta)	0.085 s for the 3,375-line delta (≈ 66× less than the full rebuild); the result reconciles: summary total 249,441,590.64 = fact total 249,441,590.64	**Adequate and scalable.** Cost follows the size of the delta, not of the history.

Summary: the *model* (grain, conformed dimensions, surrogate keys) survived the change. Three parts failed: the product-code mapping, the delivery partner attribute, and the Type 1 loyalty tier. The *pipeline* (nightly full rebuild of aggregates) fails as volume grows.

C. Adaptation: the revised design

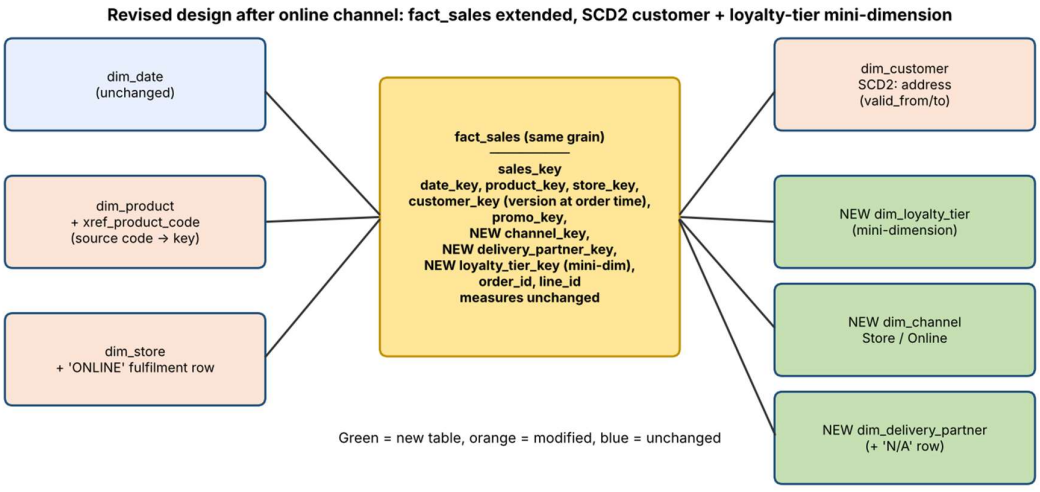


Figure 11.1 – Revised star schema after the online channel (green = new, orange = modified, blue = unchanged)

C.1 Schema changes (additive only, so there is no redesign of existing structures)

```
-- New conformed / junk-style dimensions
CREATE TABLE dim_channel          (channel_key INTEGER PRIMARY KEY, channel_name VARCHAR(10));
```

```
INSERT INTO dim_channel VALUES (1,'Store'),(2,'Online');
CREATE TABLE dim_delivery_partner (delivery_partner_key INTEGER PRIMARY KEY, partner_name
VARCHAR(30));
INSERT INTO dim_delivery_partner VALUES (0,'Not
applicable'),(1,'SwiftShip'),(2,'RoadRunner'),(3,'QuickDrop'),(4,'CityCourier');

-- Tier as a mini-dimension (a handful of rows), referenced from the fact
CREATE TABLE dim_loyalty_tier (loyalty_tier_key INTEGER PRIMARY KEY, tier_name VARCHAR(10));

-- Online fulfilment pseudo-store and a crosswalk for the new product codes
INSERT INTO dim_store(store_key,store_code,city_name,state_name,region_name) VALUES
(121,'ONLINE','Online Fulfilment','Online','Online');
CREATE TABLE xref_product_code(source_system VARCHAR(10), source_code VARCHAR(20), product_key
INTEGER,
                              PRIMARY KEY (source_system, source_code));

-- Existing fact rows get sensible defaults, so no data is rewritten or reloaded
ALTER TABLE fact_sales ADD COLUMN channel_key          INTEGER DEFAULT 1;   -- Store
ALTER TABLE fact_sales ADD COLUMN delivery_partner_key INTEGER DEFAULT 0;   -- Not applicable
ALTER TABLE fact_sales ADD COLUMN loyalty_tier_key     INTEGER;             -- tier at order time
```

Applying these structural changes to the 1M-row warehouse took 0.13 s, including creating and filling the crosswalk, because adding a column with a default does not rewrite existing rows.

C.2 Customer history: address as SCD Type 2, tier as a mini-dimension

- **Address, Type 2** (unchanged technique, now applied to online customers too). The fact stores the customer_key of the version valid on the order date (the date-range lookup of Task 5).
- **Loyalty tier: moved from Type 1 to history-preserving.** I carry the tier **in the fact as a foreign key to a small mini-dimension** (4 rows), rather than only as an extra Type 2 column in dim_customer. A customer who changes tier often would otherwise generate a new customer version row each time. With 40% of customers changing tier a year, that is up to about 40% more rows annually in a customer dimension that will have millions of online customers. The mini-dimension keeps dim_customer growing only with address changes. (The simulation applied tier and address changes as Type 2 versions for simplicity: 2,000 changes cost 0.03 s and +2,000 rows. The mini-dimension reduces this growth in production.)

C.3 ETL: hourly micro-batch with incremental aggregates

1. **Extract** every hour from the online source (watermark on order_ts) into the staging area, together with the store feed; each batch carries a batch_id.
2. **Conform:** map ONL- product codes to product_key through xref_product_code; unmapped codes go to an inferred ‘Unknown product’ member with an alert (late-arriving member); apply the Task 5 cleaning rules (currency, dates, duplicates).
3. **Dimensions first:** customer (SCD Type 2 on address; tier changes through the mini-dimension), then new delivery partners and channel.
4. **Load facts** with `INSERT … SELECT`, using the customer version valid on the order date. Idempotent through the UNIQUE(order_id, line_id) constraint.
5. **Merge the delta into the summary tables** (an upsert) instead of rebuilding them:

```
INSERT INTO agg_qtr_region_cat(region_name, category_name, year_quarter, net_amount, qty)
SELECT s.region_name, p.category_name, d.year_quarter, SUM(f.net_amount), SUM(f.quantity)
FROM fact_sales f JOIN dim_date d ON d.date_key=f.date_key JOIN dim_product p ON
p.product_key=f.product_key
     JOIN dim_store s ON s.store_key=f.store_key
WHERE f.sales_key > :last_loaded_key                    -- only this batch
GROUP BY 1,2,3
ON CONFLICT (region_name, category_name, year_quarter)
DO UPDATE SET net_amount = net_amount + excluded.net_amount, qty = qty + excluded.qty;
```

6. **Audit and publish:** read = loaded + rejected, and summary total = fact total, then update the ‘data as of’ timestamp shown on the dashboards.

Order-count measures. The order_count in the promotion × store summary can be incremented only if an order never spans two batches. The extract therefore takes **complete orders only** (orders with all lines).

D. Evidence-based defence

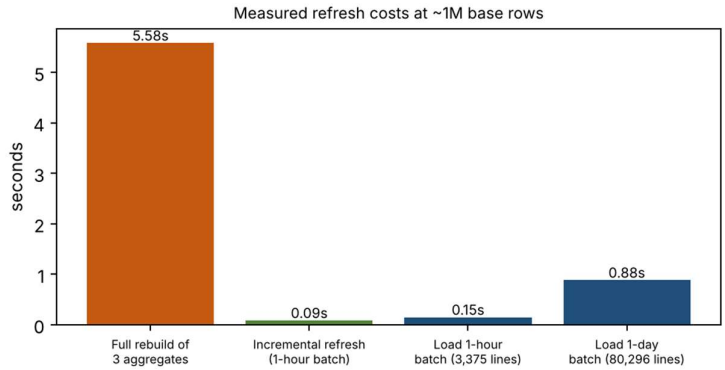


Figure 11.2 – Measured cost of the hourly refresh steps on the 1,000,000-row warehouse

Claim	Measurement / calculation
The star model is retained, so no redesign	Grain and measures unchanged. Online lines loaded into the same fact table (83,671 online lines added to 1,000,000 store lines). Online net sales loaded: 28,998,965.13; store net sales unchanged at 248,282,422.18.
Schema changes are cheap and low-risk	0.13 s for all DDL, with no rewrite of the existing 1,000,000 rows. Dimension changes are additive.
Hourly load fits comfortably	1-hour batch: 0.15 s. A full day (80,296 lines): 0.88 s. That is about 0.004% of the hourly window for one hour of data. Peak-hour store data would need to be several hundred times larger before the load took more than a minute.
Incremental refresh is necessary and sufficient	Full rebuild 5.58 s (grows linearly, so the one-hour window is exceeded at about 600 million rows); incremental merge 0.085 s (66× faster). Correctness check: incremental summary total 249,441,590.64 = fact total 249,441,590.64.
History at order time is now correct	Type 1 tier would misattribute 9.98% of historic net sales (2,000 of 20,000 customers changed tier). Type 2 versions fix this at a cost of 0.028 s and +10% dimension rows for that change volume.
Dashboards stay fast	Dashboard queries read summary tables (Q1: about 1 ms at 1M rows versus 1368 ms from the fact). Freshness lag is the hourly schedule plus about 1 s of processing.
Storage	The test warehouse is 143 MB after adding the online channel, the new dimensions and extra columns. Summary tables are about 0.14 MB.

What I would monitor after deployment

- **Freshness:** time from order creation to appearance on the dashboard (target under 70 minutes) and batch duration against the hour. An alert if a batch takes more than 50% of the interval.
- **Data quality:** rate of unmapped product codes (target 0), rejected-row rate per batch, ‘Unknown’ member rate on facts, row-count audit failures, summary-vs-fact reconciliation difference (target 0.00).
- **Dimension growth:** versions per customer per month (SCD Type 2 growth). If dim_customer growth outpaces the forecast, move more attributes into mini-dimensions.
- **Query performance:** p95 dashboard latency, and how many queries fall back to the fact table (a sign that a cuboid is missing).

- **Fact table growth and storage:** about 29M rows per year from online alone. Trigger for monthly partitioning or a columnar engine: fact table above about 100 million rows or the full-scan query above the interactive threshold.
- **Delivery-partner and channel mix:** new partners appear as new dimension members. Unexpected values signal a feed change.

Re-evaluation trigger: if the online order rate grows past the point at which a single hour's incremental batch takes more than about 10 minutes, or if sub-hourly dashboards are requested, move to a streaming (CDC) pipeline, since the model itself needs no change.

Appendix A – OLAP application source code (olap_app.py)

Run with `python olap_app.py --size 1000000` after building the databases with `python freshcart_dw.py build 1000000`. The full file is also provided as a separate download.

```
"""FreshCart OLAP console application (SQL + Python).
Run:  python olap_app.py --size 100000
Commands (type 'help'):  show | measure <net|qty|gross> | rollup <dim> | drilldown <dim> | slice
<dim> <level> <value>
                         dice <dim> <level> <v1,v2,..> | pivot <rowdim> <coldim> | reset | sql |
reconcile | quit
dim = time | product | store | customer
"""
import sqlite3, argparse, os, sys
import pandas as pd

W = os.path.join(os.path.dirname(os.path.abspath(__file__)), "work")
HIER = {"time": ["ALL", "year", "quarter", "month", "day"],
        "product": ["ALL", "department", "category", "brand", "product"],
        "store": ["ALL", "region", "state", "city", "store"],
        "customer": ["ALL", "segment"]}
COL = {"time": dict(year="d.year_no", quarter="d.year_quarter", month="d.year_month",
day="d.full_date"),
       "product": dict(department="p.department_name", category="p.category_name",
brand="p.brand_name", product="p.product_name"),
       "store": dict(region="s.region_name", state="s.state_name", city="s.city_name",
store="s.store_code"),
       "customer": dict(segment="c.segment")}
JOIN = {"time": ("d", "JOIN dim_date d ON d.date_key=f.date_key"),
        "product": ("p", "JOIN dim_product p ON p.product_key=f.product_key"),
        "store": ("s", "JOIN dim_store s ON s.store_key=f.store_key"),
        "customer": ("c", "JOIN dim_customer c ON c.customer_key=f.customer_key")}
MEAS = {"net": "f.net_amount", "qty": "f.quantity", "gross": "f.gross_amount"}



class Cube:
    def __init__(self, size):
        self.conn = sqlite3.connect(os.path.join(W, f"star_{size}.db")); self.size = size
        self.reset()

    def reset(self):
        self.level = {"time": "quarter", "product": "category", "store": "region", "customer":
"ALL"}
        self.filters = []          # (dim, level, [values])
        self.measure = "net"; self.last_sql = ""; self.pivot_axes = None

    def build_sql(self, level=None, filters=None):
        level = level or self.level; filters = self.filters if filters is None else filters
        gdims = [d for d in level if level[d] != "ALL"]
        cols = [COL[d][level[d]] for d in gdims]
        used = set(gdims) | {d for d, _, _ in filters}
        joins = " ".join(JOIN[d][1] for d in ["time", "product", "store", "customer"] if d in used)
        where, params = [], []
        for d, lv, vals in filters:
            where.append(f"{COL[d][lv]} IN ({','.join('?' * len(vals))})"); params += vals
        sql = f"SELECT {', '.join(cols + [f'ROUND(SUM({MEAS[self.measure]}),2) AS
{self.measure}'])} FROM fact_sales f {joins}"
        if where: sql += " WHERE " + " AND ".join(where)
        if cols: sql += " GROUP BY " + ", ".join(cols) + " ORDER BY " + ", ".join(cols)
        return sql, params, gdims

    def run(self, **kw):
        sql, params, gdims = self.build_sql(**kw)
        self.last_sql = sql.replace("?", "{}").format(*[repr(p) for p in params]) if params else
sql
        return pd.read_sql_query(sql, self.conn, params=params), gdims

    # --- OLAP operations
    def rollup(self, d):
        i = HIER[d].index(self.level[d])
        if i == 0: return "already at ALL (apex) for this dimension"
```

```
        self.level[d] = HIER[d][i - 1]; return f"roll-up {d}: now at level '{self.level[d]}'"

    def drilldown(self, d):
        i = HIER[d].index(self.level[d])
        if i == len(HIER[d]) - 1: return "already at the lowest level"
        self.level[d] = HIER[d][i + 1]; return f"drill-down {d}: now at level '{self.level[d]}'"

    def slice(self, d, lv, v):
        self.filters.append((d, lv, [v])); self.level[d] = "ALL"; return f"slice {d}.{lv} = {v}
(dimension removed from grouping)"

    def dice(self, d, lv, vals):
        self.filters.append((d, lv, vals)); return f"dice {d}.{lv} IN {vals}"

    def pivot(self, a, b):
        self.pivot_axes = (a, b); return f"pivot rows={a} columns={b}"

    def show(self):
        df, g = self.run()
        if self.pivot_axes and len(g) >= 2:
            a, b = self.pivot_axes
            ca, cb = COL[a][self.level[a]].split(".")[1], COL[b][self.level[b]].split(".")[1]
            df = df.pivot_table(index=ca, columns=cb, values=self.measure, aggfunc="sum",
fill_value=0, margins=True, margins_name="TOTAL")
        return df.head(25).to_string()

    def reconcile(self):
        out = []
        df, g = self.run(); view_total = round(float(df[self.measure].sum()), 2)
        flt_total = self.conn.execute(self.build_sql(level={d: "ALL" for d in HIER})[0],
self.build_sql(level={d: "ALL" for d in HIER})[1]).fetchone()[0]
        out.append(f"view total {view_total:,.2f}  vs  same filters at apex cuboid {flt_total:,.2f}
-> {'OK' if abs(view_total - flt_total) < 0.05 else 'MISMATCH'}")
        o = sqlite3.connect(os.path.join(W, f"oltp_{self.size}.db"))
        src = o.execute("SELECT ROUND(SUM(qty*unit_price-discount_amt),2), COUNT(*) FROM
order_lines").fetchone()
        raw = pd.read_sql_query("SELECT qty,unit_price,discount_amt FROM order_lines", o)
        py = round(float((raw.qty * raw.unit_price - raw.discount_amt).sum()), 2)
        star = self.conn.execute("SELECT ROUND(SUM(net_amount),2), COUNT(*) FROM
fact_sales").fetchone()
        out.append(f"unfiltered net sales: star {star[0]:,.2f} ({star[1]} rows) | OLTP SQL
{src[0]:,.2f} ({src[1]} rows) | pandas {py:,.2f}")
        out.append("row counts and grand totals reconcile -> OK" if star[1] == src[1] and
abs(star[0] - src[0]) < 0.05 and abs(py - src[0]) < 1 else "RECONCILIATION FAILED")
        return "\n".join(out)



def main():
    ap = argparse.ArgumentParser(); ap.add_argument("--size", type=int, default=100000); a =
ap.parse_args()
    cube = Cube(a.size); tty = sys.stdin.isatty()
    print(f"FreshCart OLAP - star schema with {a.size:,} sales lines. Type 'help'.")
    while True:
        try: line = input("olap> ") if tty else sys.stdin.readline()
        except EOFError: break
        if not tty:
            if not line: break
            print("olap>", line.strip())
        p = line.strip().split(None, 3)
        if not p: continue
        c = p[0].lower()
        try:
            if c in ("quit", "exit"): break
            elif c == "help": print(__doc__)
            elif c == "reset": cube.reset(); print("reset to default view")
            elif c == "measure": cube.measure = p[1]; print("measure =", p[1])
            elif c == "rollup": print(cube.rollup(p[1])); print(cube.show())
            elif c == "drilldown": print(cube.drilldown(p[1])); print(cube.show())
            elif c == "slice": print(cube.slice(p[1], p[2], p[3])); print(cube.show())
            elif c == "dice": print(cube.dice(p[1], p[2], p[3].split(","))); print(cube.show())
            elif c == "pivot": print(cube.pivot(p[1], p[2])); print(cube.show())
            elif c == "show": print(cube.show())
            elif c == "sql": print(cube.last_sql)
```

```
            elif c == "reconcile": print(cube.reconcile())
            else: print("unknown command")
        except Exception as e:
            print("error:", e)



if __name__ == "__main__":
    main()
```

Appendix B – ETL, data generation and benchmark (excerpts from freshcart_dw.py)

The full scripts (`freshcart_dw.py`, `task11.py`) are provided as separate downloads. The fact load from the normalised OLTP tables into the star (initial load) is:

```
INSERT INTO fact_sales(order_id,line_id,date_key,product_key,store_key,customer_key,promo_key,
                       quantity,gross_amount,discount_amount,net_amount)
SELECT l.order_id, l.line_id, CAST(replace(o.order_date,'-','') AS INT),
       l.product_id, o.store_id, o.customer_id, o.promo_id,
       l.qty, ROUND(l.qty*l.unit_price,2), l.discount_amt, ROUND(l.qty*l.unit_price-
l.discount_amt,2)
FROM order_lines l JOIN orders o USING(order_id);

-- Benchmark queries (Task 8), star versions
-- Q1
SELECT s.region_name, p.category_name, d.year_quarter, ROUND(SUM(f.net_amount),2)
FROM fact_sales f JOIN dim_date d ON d.date_key=f.date_key JOIN dim_product p ON
p.product_key=f.product_key
     JOIN dim_store s ON s.store_key=f.store_key GROUP BY 1,2,3;
-- Q2
SELECT p.promo_name, s.state_name, ROUND(AVG(t.basket),2) FROM
  (SELECT order_id, promo_key, store_key, SUM(net_amount) AS basket FROM fact_sales GROUP BY
order_id) t
  JOIN dim_promotion p ON p.promo_key=t.promo_key JOIN dim_store s ON s.store_key=t.store_key GROUP
BY 1,2;
-- Q3
SELECT p.department_name, dd.day_type, ROUND(SUM(f.net_amount),2)
FROM fact_sales f JOIN dim_date dd ON dd.date_key=f.date_key JOIN dim_product p ON
p.product_key=f.product_key GROUP BY 1,2;

-- Summary tables built after the load
CREATE TABLE agg_qtr_region_cat  AS SELECT s.region_name, p.category_name, d.year_quarter,
SUM(f.net_amount) AS net_amount, SUM(f.quantity) AS qty FROM ... GROUP BY 1,2,3;
CREATE TABLE agg_promo_store     AS SELECT promo_key, store_key, SUM(net_amount) AS net_amount,
COUNT(DISTINCT order_id) AS order_count FROM fact_sales GROUP BY 1,2;
CREATE TABLE agg_day_type_dept   AS SELECT p.department_name, d.day_type, SUM(f.net_amount) AS
net_amount FROM ... GROUP BY 1,2;
```